

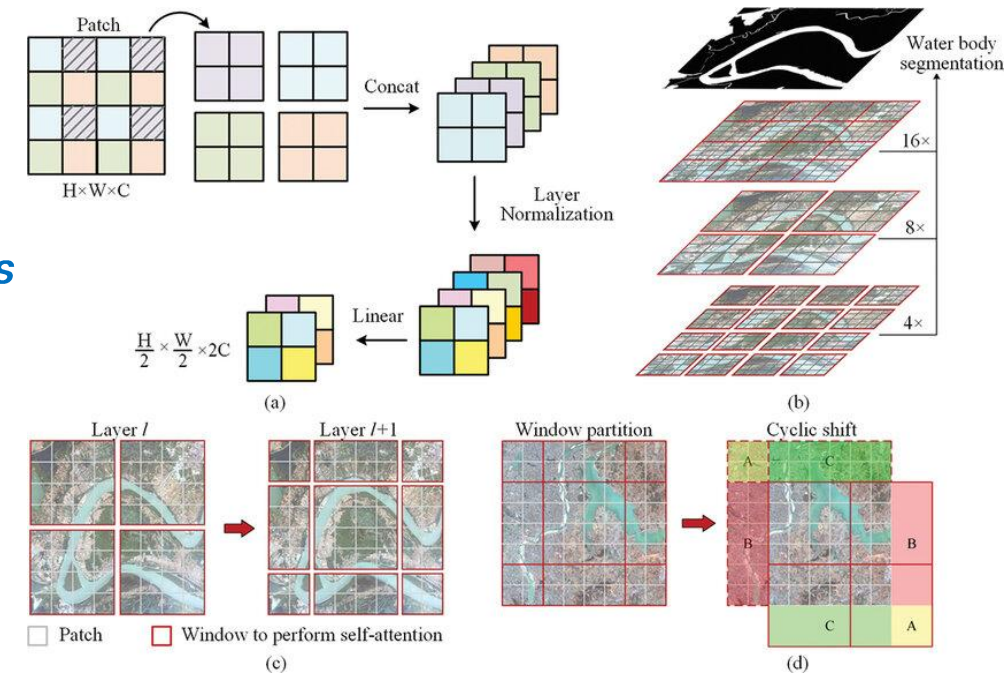
ViT & Swin Transformer

AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE

"한 이미지는 16x16 단어의 가치가 있다: 대규모(확장 가능한) 이미지 인식을 위한 트랜스포머"
"이미지를 16x16(기본 구조) 크기의 패치로 나누어 각각을 단어(token)처럼 취급"



Swin Transformer: Hierarchical Vision Transformer using Shifted Windows



| Table of Contents

- I. ViT Intro
- II. A Visual Guide to Vision Transformers
- III. ViT 세부 구조 및 특징
- IV. ViT 실험 결과 및 한계
- V. SwinT Intro
- VI. Architecture of SwinT
- VII. SwinT 실험 결과 및 한계
- VIII. Questions

Chapter |

ViT Intro

1.1 Introduction

Can Attention/Transformers be used from more than text processing?

1.1 Introduction

ViLBERT: A Visolinguistic Transformer



pop artist performs at the
festival in a city.



a worker helps to clear
the debris.

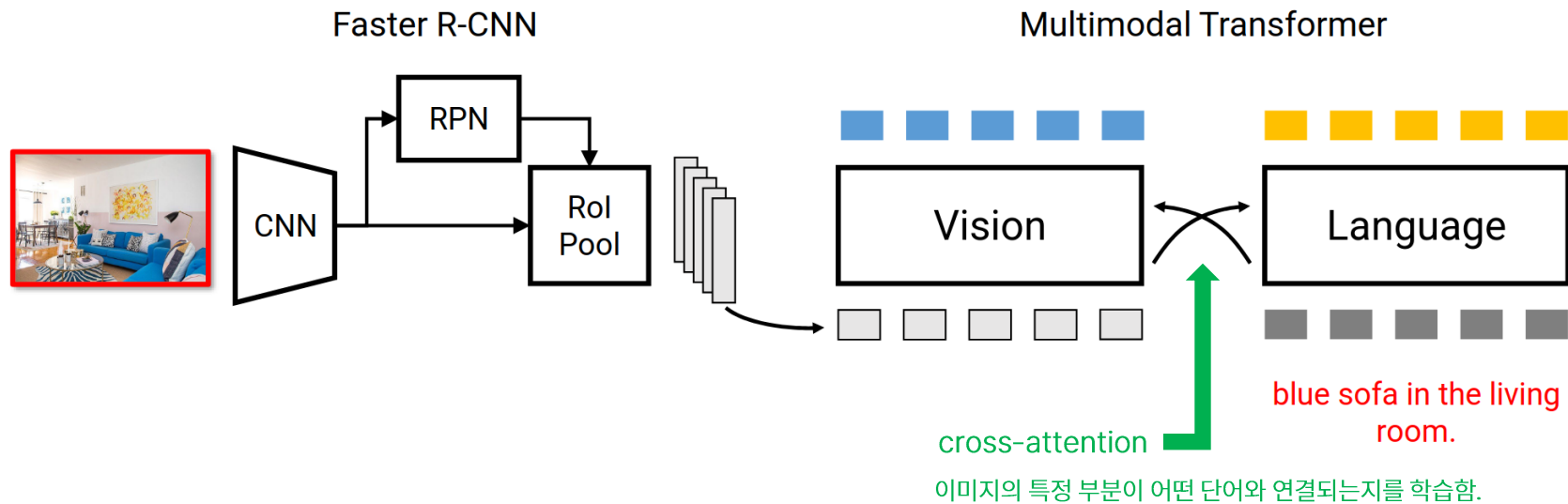


blue sofa in the living
room.

Image and captions from: Sharma, Piyush, et al. "Conceptual captions: A cleaned, hypernymed, image alt-text dataset for automatic image captioning." ACL. 2018.

1.1 Introduction

ViLBERT: A Visiolinguistic Transformer



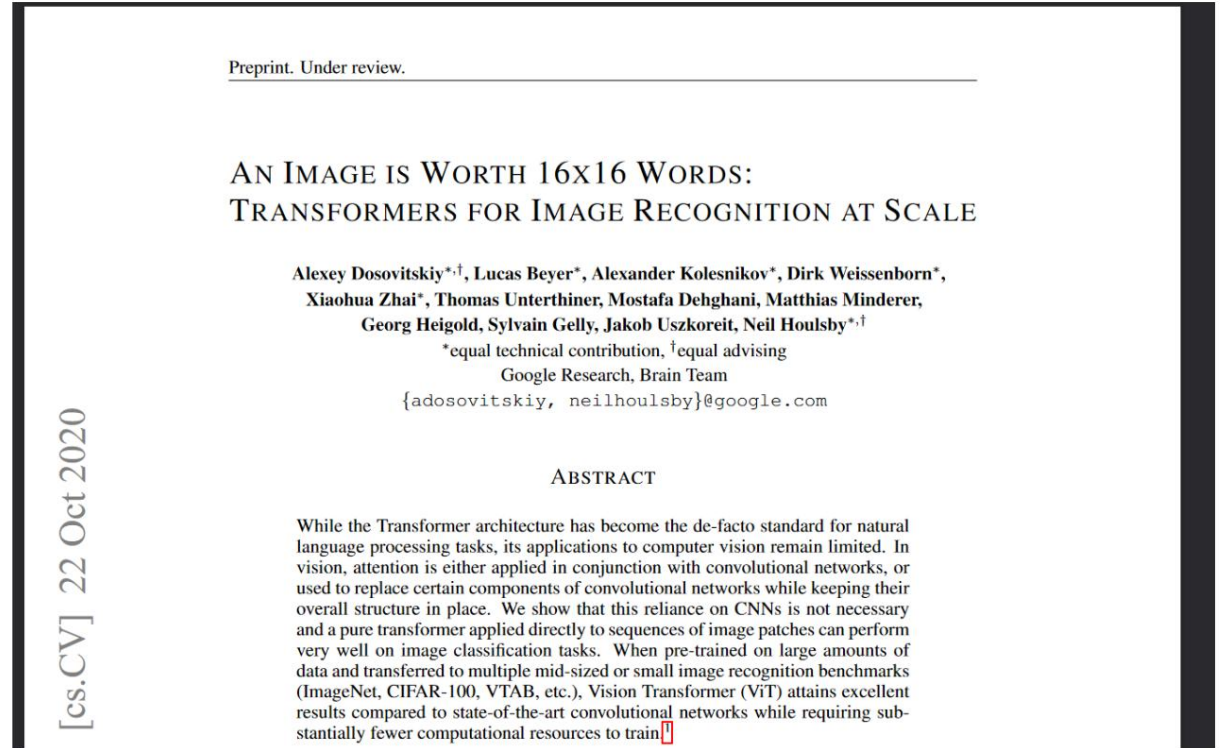
Lu et al "Vilbert: Pretraining task-agnostic visiolinguistic representations for vision-and-language tasks." *NeurIPS*. 2019.
 Ren et al. "Faster r-cnn: Towards real-time object detection with region proposal networks." *NeurIPS*. 2015.

Transformer 구조는 CV 분야에서는 convolution과 함께 사용되거나 convolution을 대체해서 사용하는 등 convolution 구조를 유지한 채 함께 사용되는 정도로만 활용 되고 있었음

1.1 Introduction

What about for just image inputs? Without Convolution?

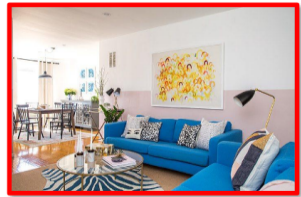
What About Vision with just Self-Attention?



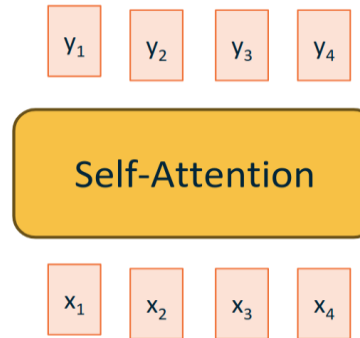
Slide progression inspired by Soheil Feizi

1.1 Introduction

What to do with images?



How should we
“tokenize” images?



- **Pixels?** Too computationally intensive $O(n^2)$!
- **Patches!**

Vision Transformers

Treat pixels as words and calculate attention!



1. Pixel 단위로 한다면:

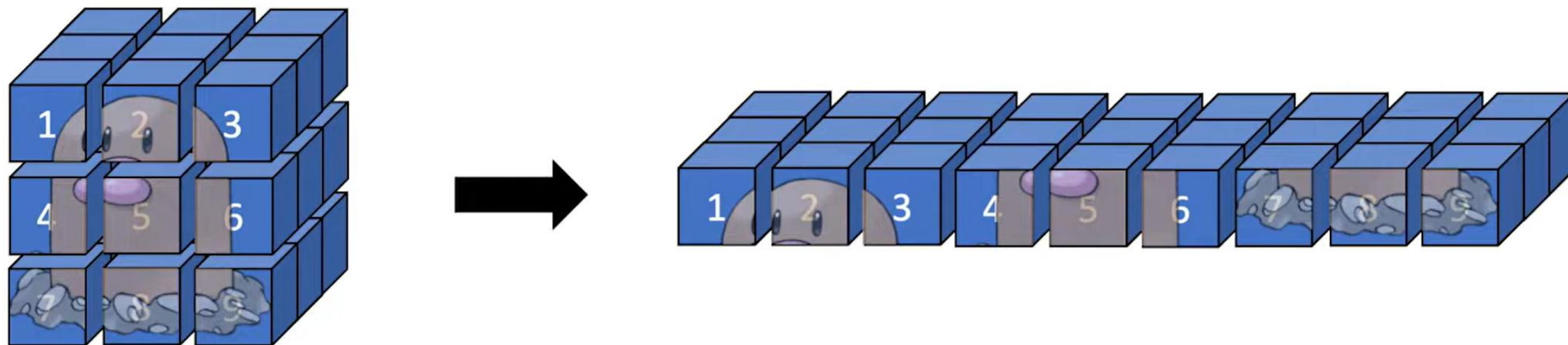
- 224×224 이미지
- 픽셀 수는 $224 \times 224 = 50,176$
- 만약 이것을 각각 토큰으로 취급해서 Self-Attention 한다면?
→ $O(n^2) = O(50,176^2) \approx 2.5$ billion 개의 연산!
- ✓ 너무 비싸고 메모리도 감당 안 됨

2. 그래서 ViT는 Patch로 자름 (예: 16×16)

- 224×224 이미지를 16×16 패치로 자르면:
→ $14 \times 14 = 196$ 개의 패치만 생성됨
- 그러면 Self-Attention의 복잡도는 $O(196^2) = 38,416$
- ✓ 비교적 계산 가능

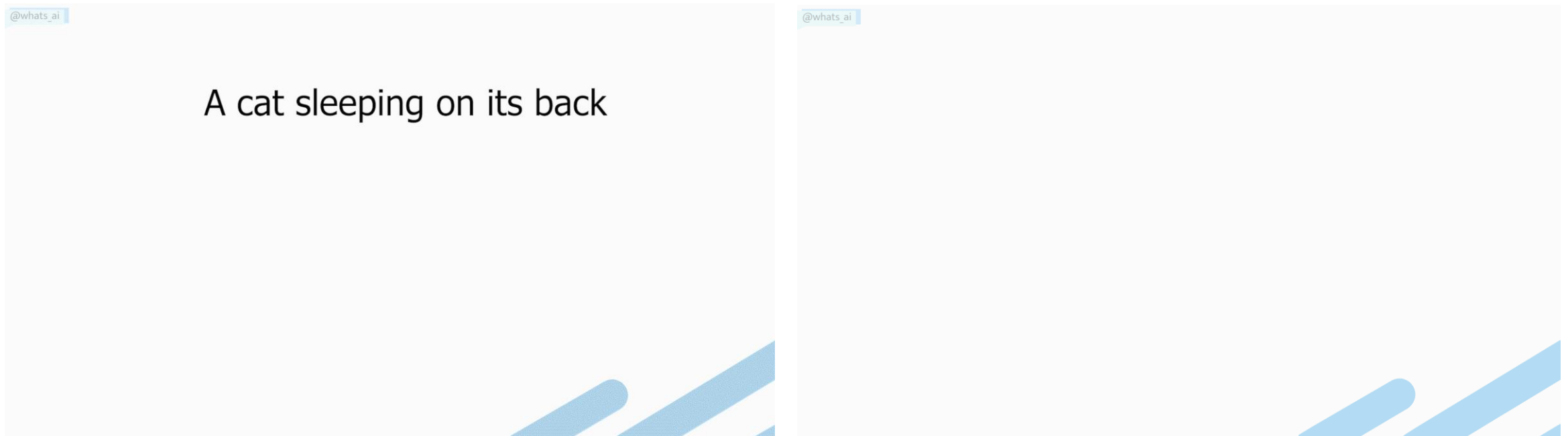
1.1 Introduction

Patches!



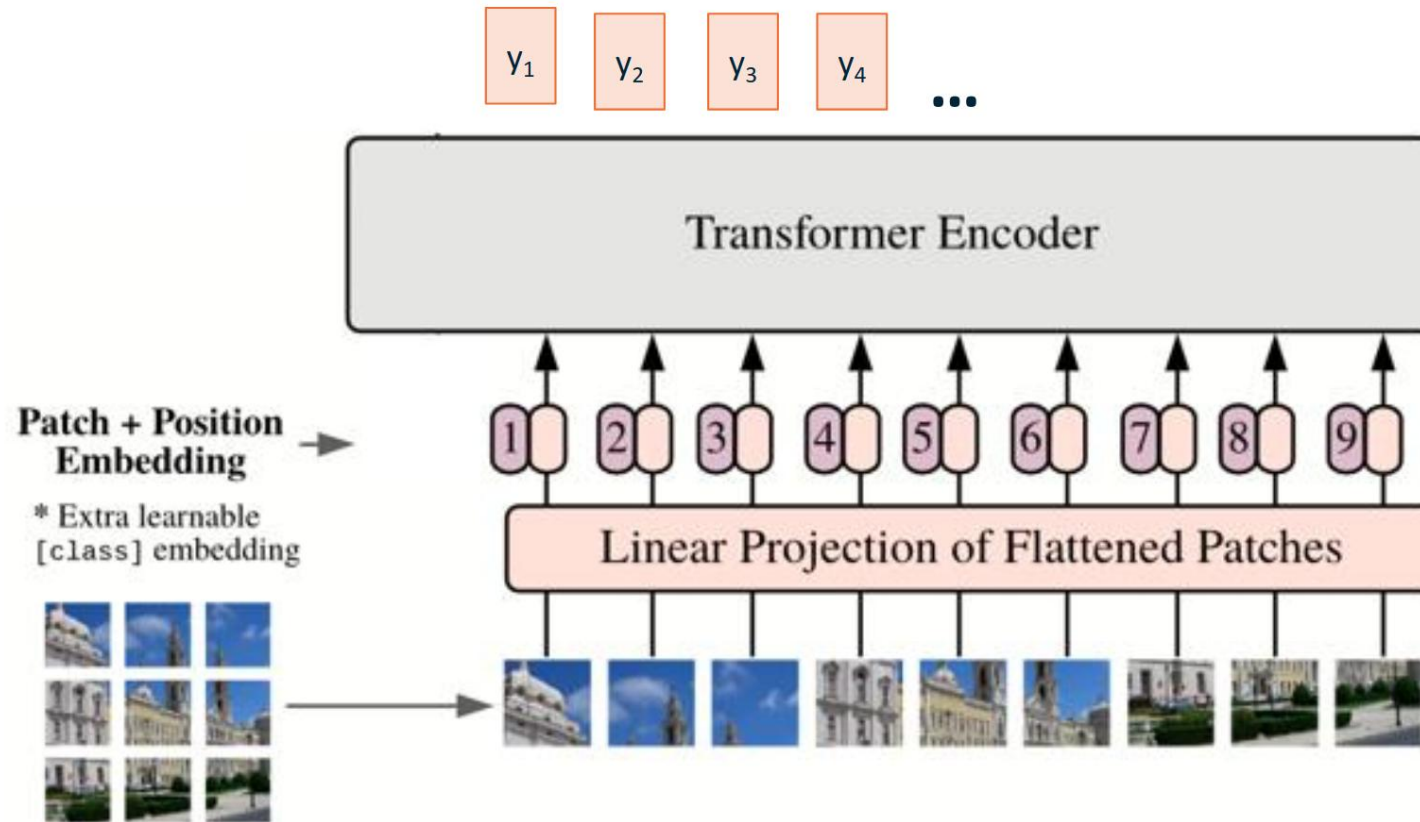
1.1 Introduction

Self-attention example in Transformers for NLP (left) and computer vision (right).



1.1 Introduction

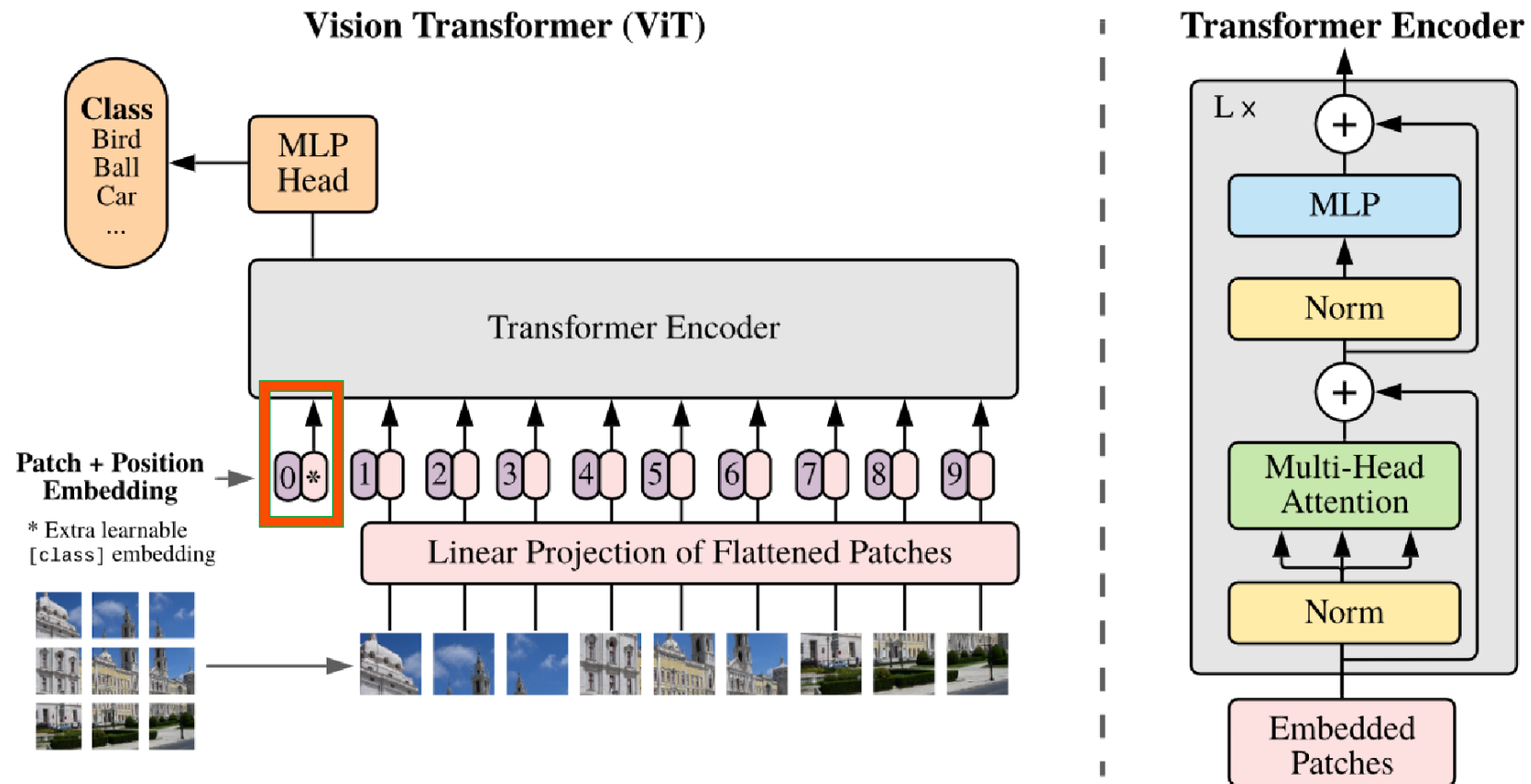
Patches as input to Self-Attention



- How do we do classification?

1.1 Introduction

Vision Transformer (ViT)



Chapter II

A Visual Guide to Vision Transformers

2.0 Lets start with the data



Mountain



Bird



Train



Castle



Lynx

- 일반적인 합성곱 신경망(CNN)과 마찬가지로 Vision Transformer도 지도 학습(Supervised Learning) 방식으로 학습. 즉, 이미지와 이에 맞는 레이블(label)로 구성된 데이터셋으로 모델이 학습

2.1 Focus on one data point



Bird

- Vision Transformer가 내부적으로 어떻게 동작하는지 알아보기 위해, 하나의 데이터(배치 크기 1)에 대해서만 먼저 집중해보겠습니다. 그리고 이 질문을 함께 생각해보시죠: Transformer에 이 데이터를 입력하기 위해서는 어떻게 준비(전처리)해야 할까요?

2.2 Forget the label for the moment



- 레이블은 나중에 더 관련성있게 살펴보겠습니다. 지금은 이미지 하나만 남겨놓고 보겠습니다.

2.3 Create patches of the image



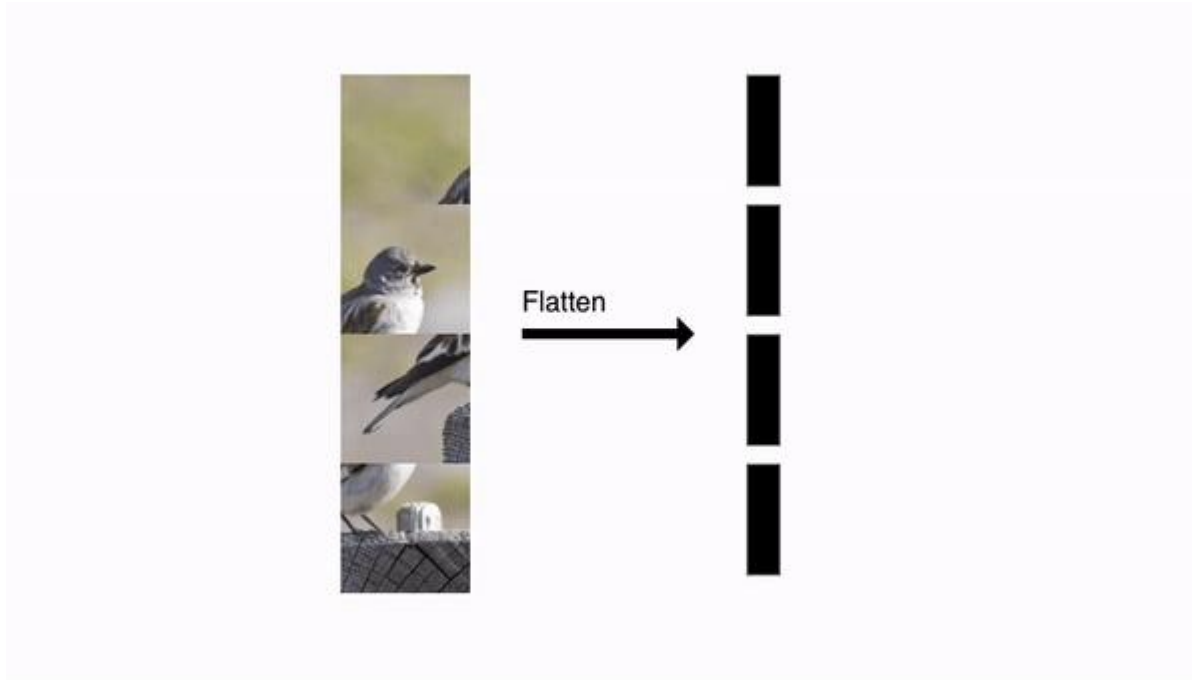
- 전체 이미지를 동일한 크기의 패치($p \times p$) 이미지로 나누어 Transformer 내부에서 사용할 수 있도록 준비합니다.

2.4 Flatting of the image patches



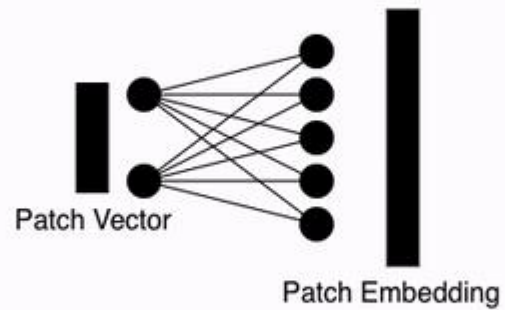
- 패치들을 $p' = p^2 \times c$ 크기의 벡터로 평탄화(flaten)합니다. 이 때 p 는 패치의 한 변의 크기이고, c 는 채널 수입니다.

2.5 Creating patch embeddings



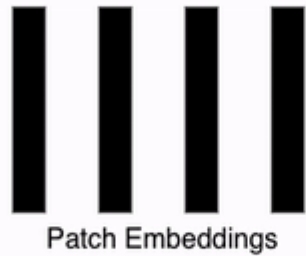
- 앞에서 이미지 패치로부터 만든 벡터들을 선형 변환을 통해 인코딩합니다. 이렇게 만들어진 패치 임베딩 벡터(Patch Embedding Vector)는 고정된 크기 d 를 갖습니다.

2.6 Embedding all patches



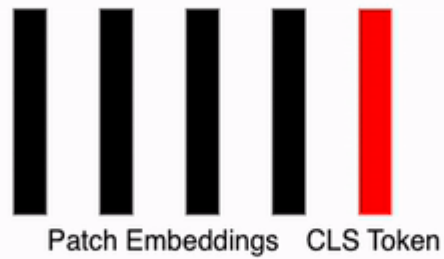
- 이미지 패치들을 모두 고정된 크기의 벡터로 임베딩하게 되면 $n \times d$ 크기의 배열을 얻게 됩니다. 여기서 n 은 이미지 패치의 개수이고, d 는 하나의 패치가 임베딩된 크기입니다.

2.7 Appending a classification token



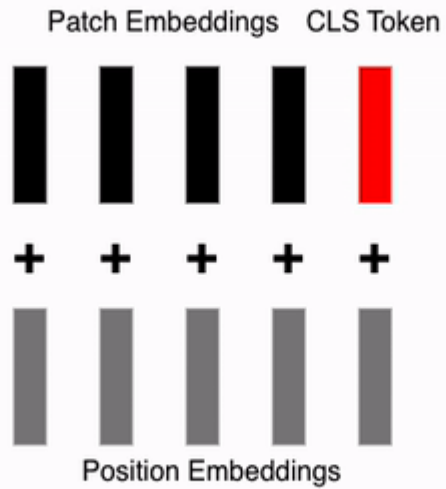
- 모델을 효과적으로 학습하기 위해, 패치 임베딩에 추가로 분류 토큰(CLS token)이라 부르는 벡터를 추가합니다. 이 벡터는 신경망을 통해 **학습 가능한 매개변수**로, 무작위로 초기화됩니다. 참고로, CLS 토큰은 하나만 있으며, 모든 데이터들에 동일한 벡터를 추가합니다. (여기까지 하게 되면 n 개의 패치 임베딩에 CLS 토큰을 더하여 $(n+1)$ 개에 각 임베딩 크기 d 인, $(n+1) \times d$ 를 갖게 됩니다.)

2.8 Add positional embedding Vectors



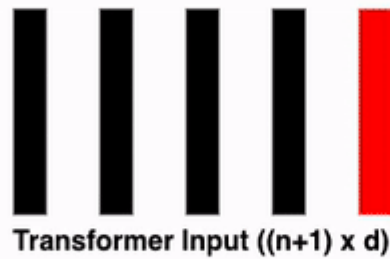
- 지금까지의 패치 임베딩에는 별도의 위치 정보가 없습니다. 모든 패치 임베딩에 **학습 가능한**, 무작위로 초기화된 위치 임베딩 벡터(Positional Embedding Vector)를 더하여 이 문제를 해결합니다. 또한, 앞에서 추가한 분류 토큰(CLS token)에도 이러한 위치 벡터를 추가합니다. (Transformer에서는 Positional Encoding의 값을 '더해'줍니다. 따라서 벡터의 크기에는 변화가 없습니다.)

2.9 Transformer Input



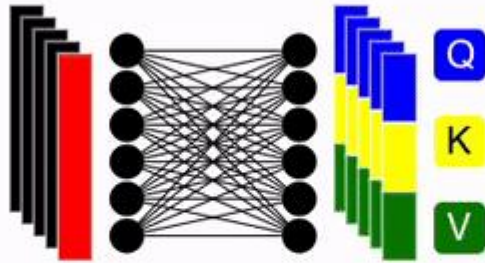
- 위치 임베딩 벡터를 추가하면 $(n+1) \times d$ 크기의 배열이 남습니다. 이 배열을 Transformer의 입력으로 제공할 것이며, 이에 대해서는 다음 단계에서 더 자세히 설명하겠습니다.

2.10.1 Transformer: QKV Creation



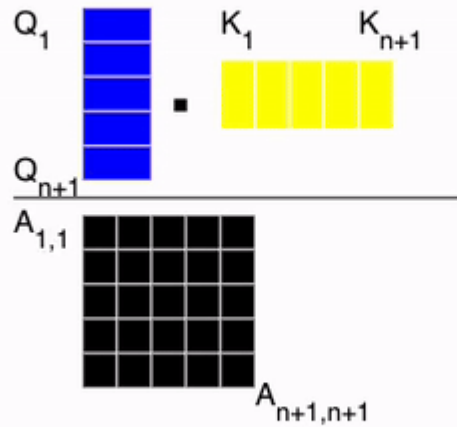
- Transformer 입력 패치 임베딩 벡터는 여러 큰 벡터에 선형적으로 임베딩됩니다. 이러한 새로운 벡터는 동일한 크기의 세 부분으로 분리됩니다. 이는 각각 Q는 쿼리(Query) 벡터, K는 키(Key) 벡터, V는 값(Value) 벡터입니다. 모든 벡터들을 (n+1)개씩 얻게 됩니다.

2.10.2 Transformer: Attention Score Calculation



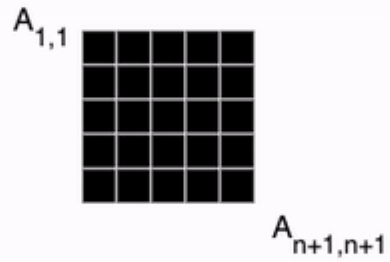
- 먼저 어텐션 스코어 A 를 계산하기 위해 모든 쿼리 벡터 Q 에 모든 키 벡터 K 를 곱합니다.

2.10.3 Transformer: Attention Score Matrix



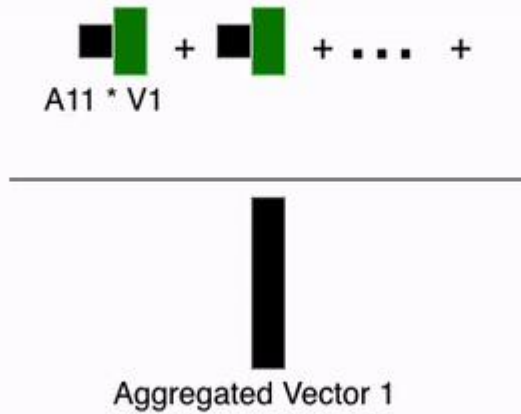
- 이렇게 얻은 어텐션 스코어 행렬 A 의 모든 행의 합이 1이 되도록 모든 행에 softmax 함수를 적용합니다.

2.10.4 Transformer: Aggregated Contextual Information Calculation



- 첫 번째 패치 임베딩 벡터에 대한 집계된 문맥 정보(aggregated contextual information)를 계산하기 위해, 어텐션 행렬의 첫 번째 행에 대해서 연산을 합니다. 여기에 값 벡터 V 의 가중치를 사용하여 첫 번째 이미지 패치 임베딩에 대한 집계된 문맥 정보 벡터(aggregated vector)를 생성합니다.

2.10.5 Transformer: Aggregated Contextual Information for every patch



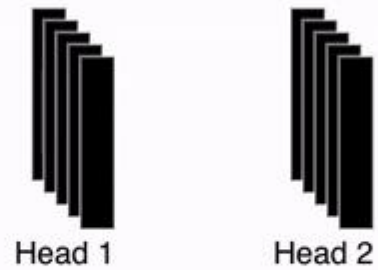
- 어텐션 스코어 행렬의 다른 행들에 대해서도 위 과정을 반복하여 $N+1$ 개의 집계된 문맥 정보 벡터를 구합니다. 즉, 모든 패치마다 하나씩 ($=N$ 개) + 분류 토큰(CLS Token)에 대해서 하나 ($=1$) 입니다. 여기까지 해서 첫번째 어텐션 헤드(Attention Head)를 구합니다.

2.10.6 Transformer: Multi-Head Attention



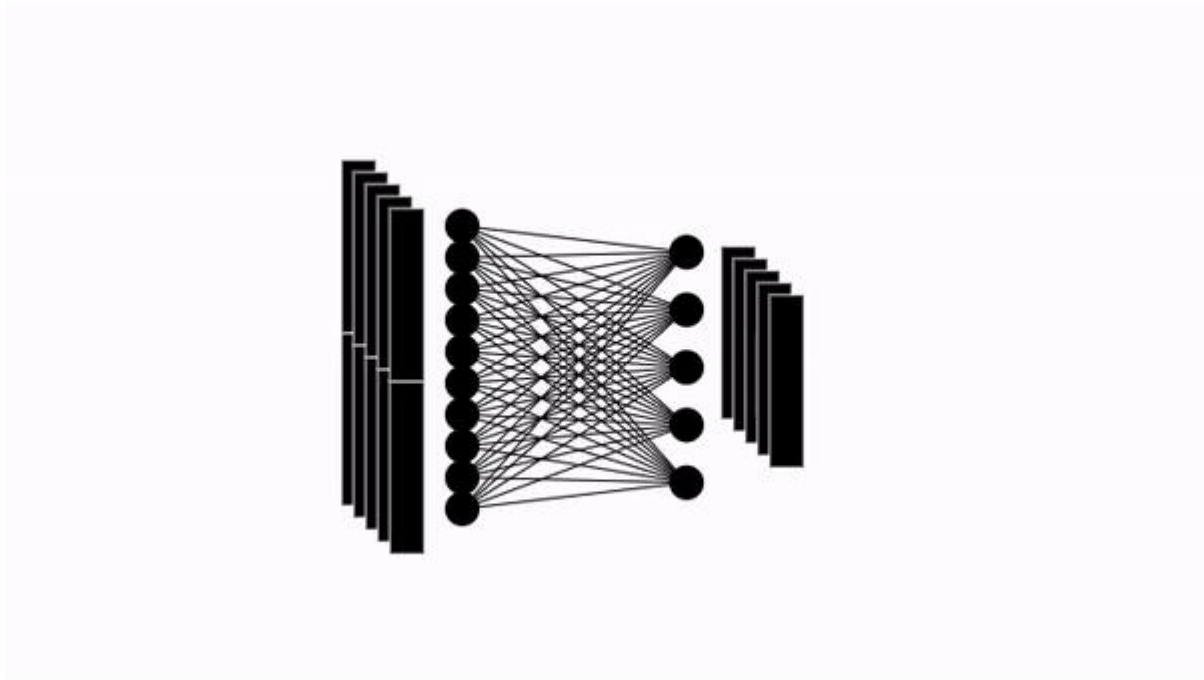
- (Transformer의) 멀티-헤드 어텐션을 다루고 있으므로, 다른 QKV들에 대해서 10.1부터 10.5까지의 전체 프로세스를 반복합니다. 위 그림에서는 2개의 헤드만 가정했지만, 일반적으로 ViT는 더 많은 헤드를 갖습니다. 이렇게 여러 개의 집계된 문맥 정보 벡터(Multiple Aggregated Contextual Information Vectors)가 생성됩니다.
- multi-head는 각 token들을 cnn에서의 필터처럼 다양한 관점에서 바라 볼 수 있도록 하는 역할

2.10.7 Transformer: Last Attention Layer Step



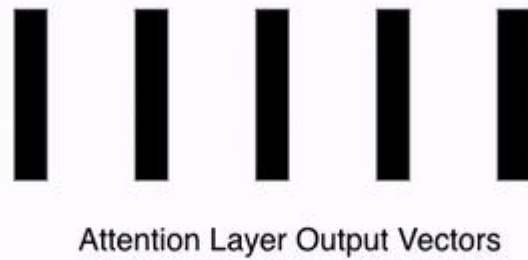
- 이렇게 생성한 여러 헤드들을 쌓은 뒤, 패치 임베딩의 크기와 같은 d 크기의 벡터로 매핑시킵니다.

2.10.8 Transformer: Attention Layer Result



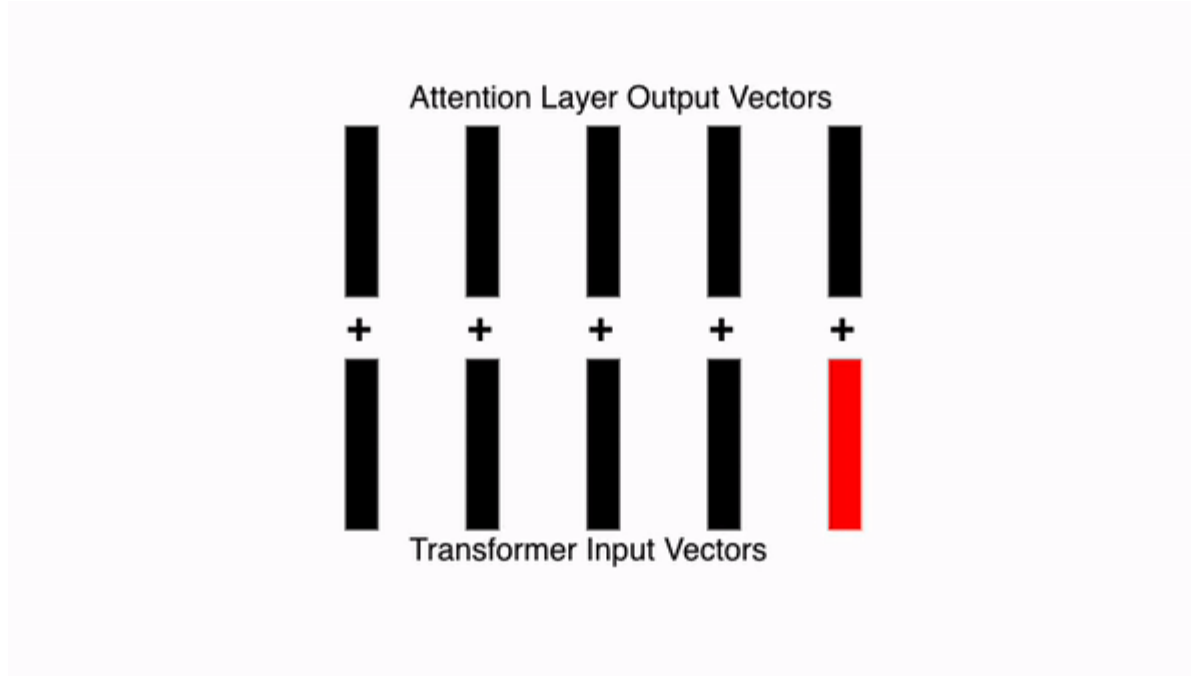
- 이렇게 이전 단계로부터 어텐션 레이어가 완성되었고, 입력 시에 사용했던 것과 정확히 같은 크기의 임베딩들을 얻었습니다.

2.10.9 Transformer: Residual connections



- Transformer에서는 잔차 연결(Residual Connection) 을 많이 사용하는데, 이것은 단순히 이전 레이어의 입력을 현재 레이어의 출력에 더해주는 것입니다. 여기서도 잔차 연결을 하겠습니다.

2.10.10 Transformer: Residual connection Result



- 이러한 잔차 연결을 통해 (동일한 크기 d 인 벡터들끼리 더하여) 같은 크기의 벡터가 생성됩니다.

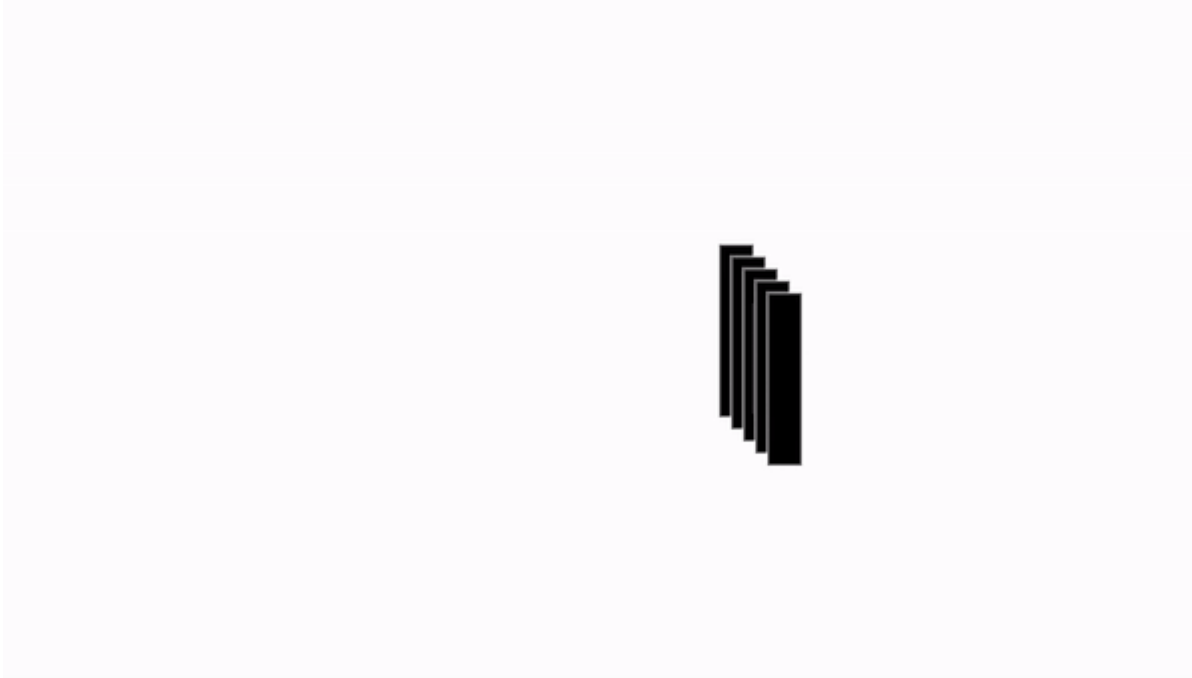
2.10.11 Transformer: Feed Forward Network



Attention Output + Transformer Input

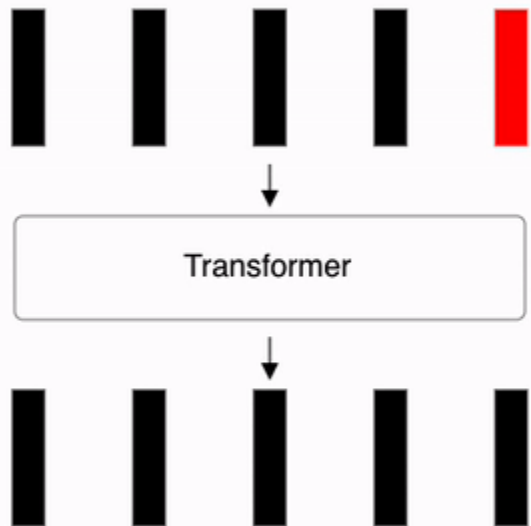
- 지금까지의 결과(output)를 비선형 활성화함수를 갖는 피드 포워드 인공 신경망에 통과시킵니다.

2.10.12 Transformer: 최종 결과 구하기 / Final Result



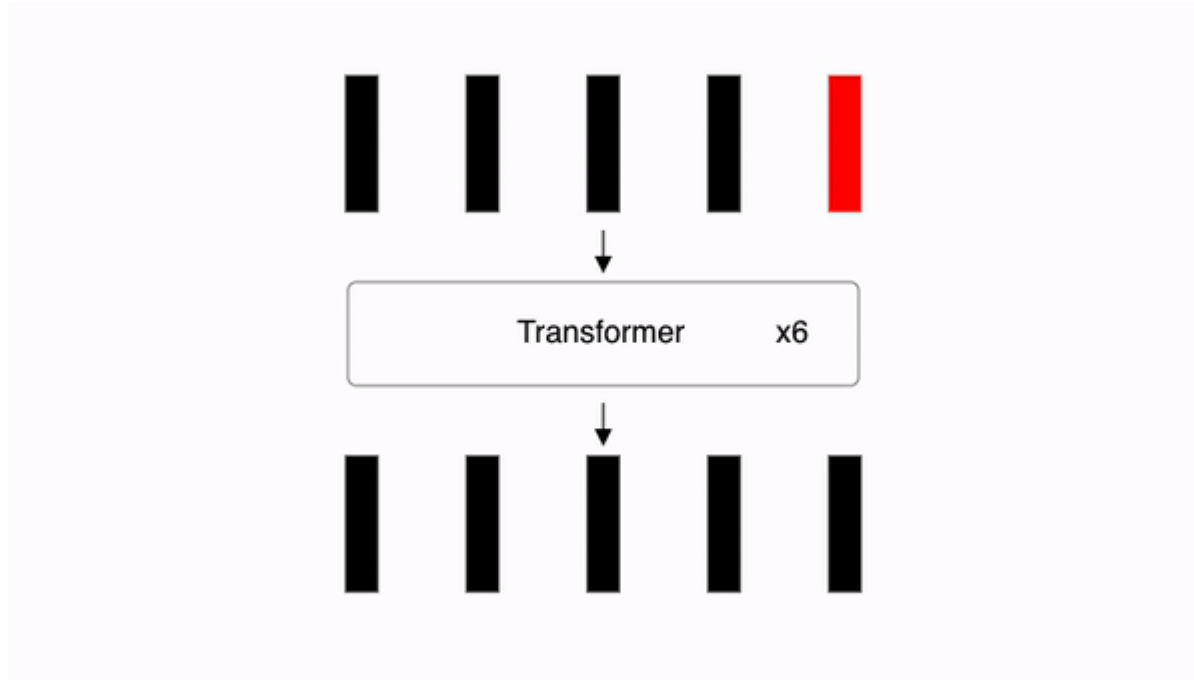
- Transformer에는 지금까지 연산 이후로 또다른 잔차 연결이 있지만, 여기서는 설명을 간소화하기 위해 건너뛰고 Transformer 레이어 연산을 마무리하겠습니다. 최종적으로 Transformer는 입력 크기와 같은 출력을 생성합니다.

2.11 Repeat Transformers



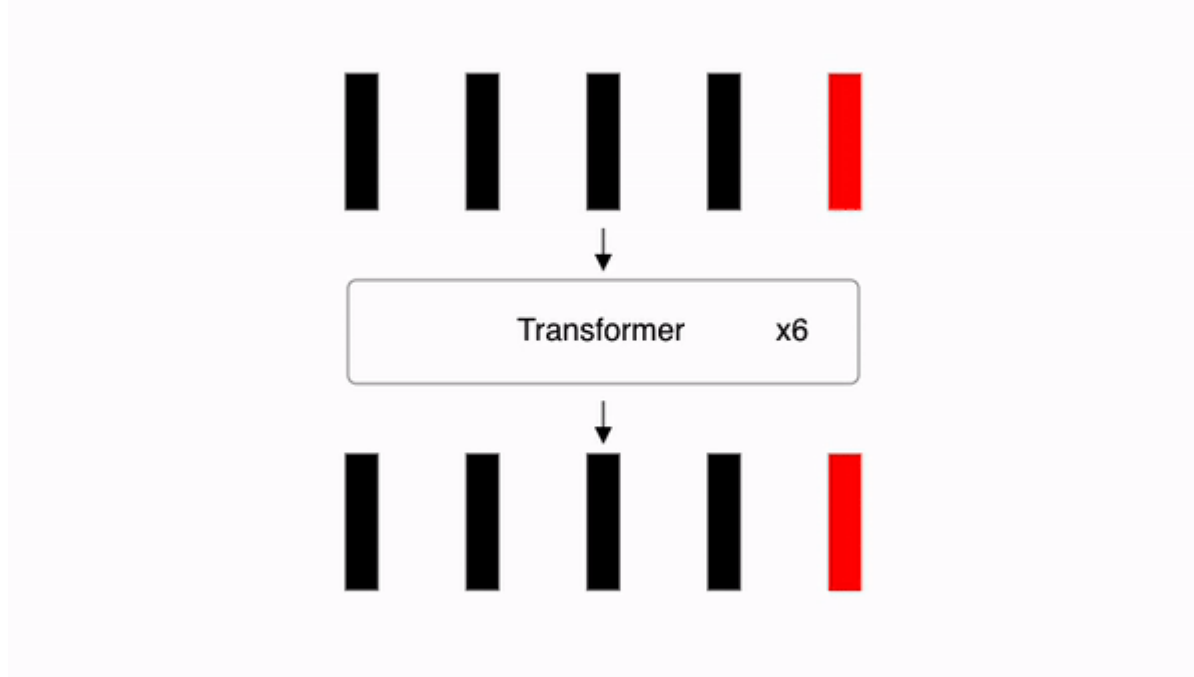
- 지금까지 진행한 10.1부터 10.12까지의 전체 Transformer 연산을 수차례 반복합니다. 여기에서는 6번을 예시로 들었습니다.

2.12 Identify Classification token output



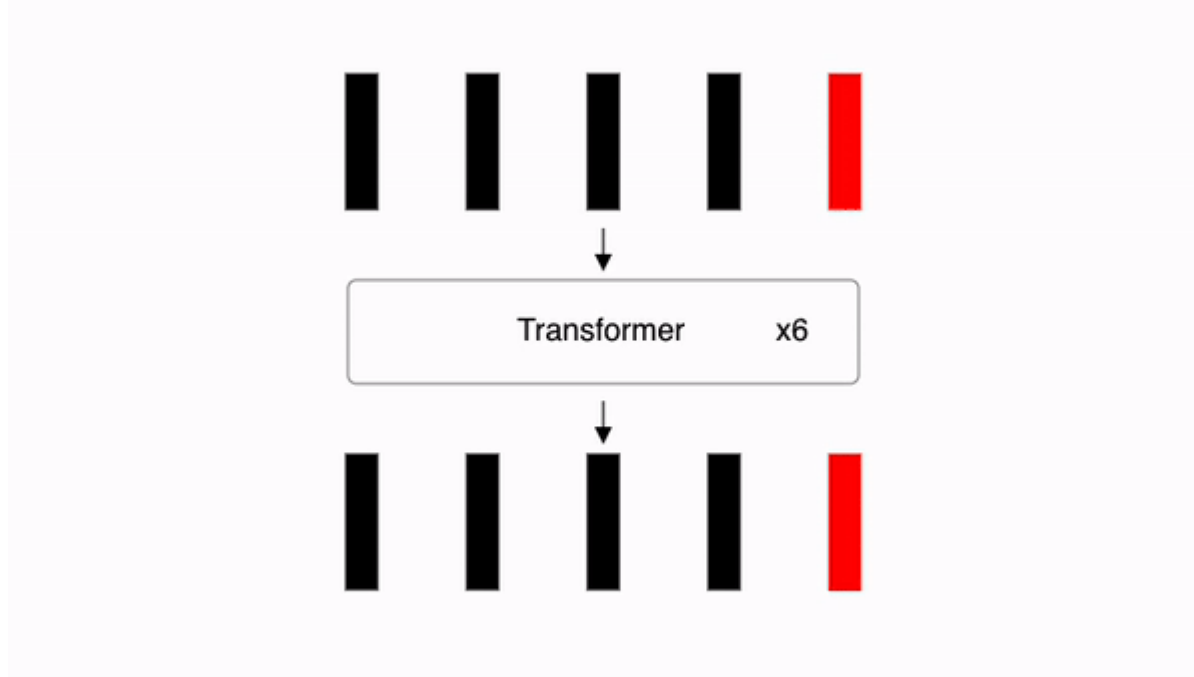
- 마지막 단계는 분류 토큰(CLS token) 출력을 확인하는 것입니다. 이 벡터는 Vision Transformer 여정의 마지막 단계에서 사용하게 됩니다.

2.13 Final Step: Predicting classification probabilities



- 최종적이고 마지막 단계에서는 분류 출력 토큰을 완전 연결(fully-connected)된 또 다른 인공 신경망에 통과시켜 입력 이미지에 대한 분류 확률(classification probabilities)을 예측합니다.

2.14 Training of the Vision Transformer



- 앞에서 예측한 분류 확률(class probabilities)과 정답(true class label)을 비교하는 표준 크로스-엔트로피 손실 함수(Cross-Entropy Loss Function)를 사용하여 Vision Transformer를 학습합니다. 모델은 역전파(backpropagation) 및 경사 하강법(gradient descent)을 사용하여 손실 함수를 최소화하는 쪽으로 모델의 가중치를 갱신하며 학습합니다.

Chapter III

ViT 세부 구조 및 특징

3.1 모델 구조에서 Normalizaion의 순서

Normalizaion의 순서 차이 (Transformer encoder vs ViT)

- Normalizaion의 순서가 attention, mlp 앞으로 온 것만 차이가 있는데 이는 transformer 후속 논문에서 앞에 쓰는 것이 transformer를 깊게 만들 때 더 효과적이라는 것이 밝혀졌기 때문.
- 이 부분이 Attention과 MLP 이전에 수행을 하기 때문에 깊은 레이어에서도 학습이 더 잘 되도록 하게 했다고 함.

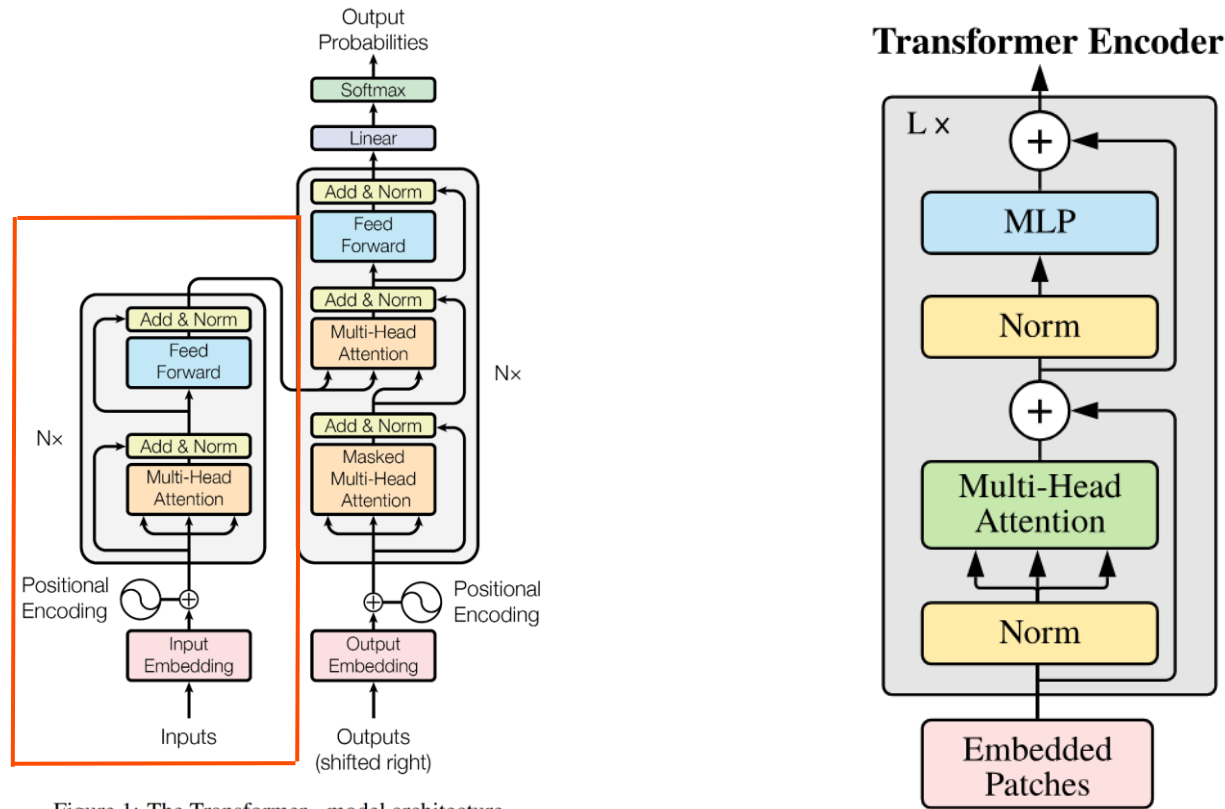


Figure 1: The Transformer - model architecture.

3.2 이미지를 패치 단위로, Sequence Data형태로

Transformer 처럼 이미지를 패치 단위로 끊어, Sequence Data형태로 만들어주었다. (Embedded Patches)

- 우선 Input 이미지를 정해진 패치 사이즈로 맞게 쪼개준다.
- 그 후 CLS Token과 Concat 시키고, 각 패치에 Positioning Embedding을 더해준다. (이는 학습되는 파라미터)



3.3 Inductive bias

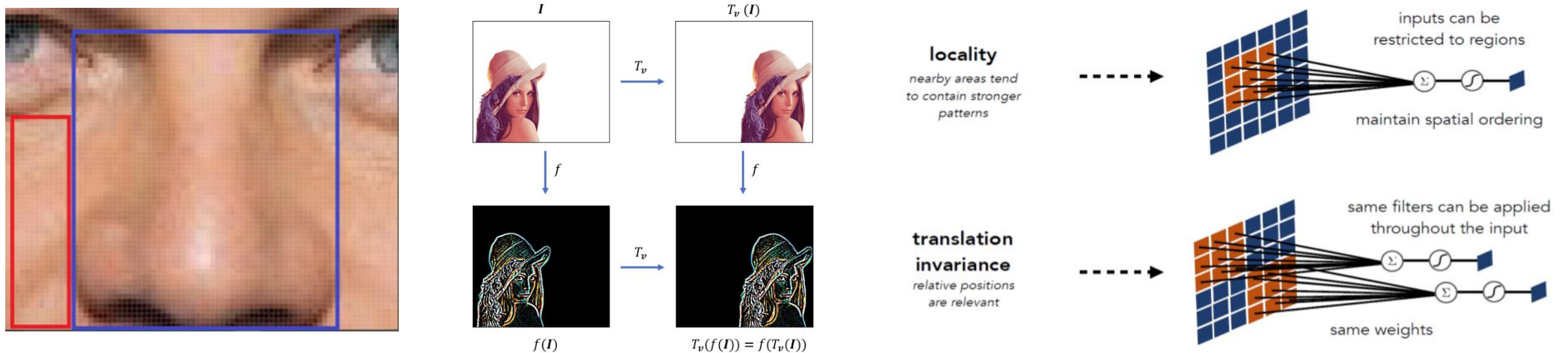
I Inductive bias (귀납적 편향)

- "a learning algorithm is the set of assumptions that the learner uses to predict outputs of given inputs that it has not encountered" - "보지 못한 input 에 대한 예측을 하기 위해 사용하는 가정의 집합".
- 머신러닝이나 딥러닝에서 학습 주체(컴퓨터, 모델)가 본 적 없는 입력(input)에 대해 출력(output)을 예측할 때 사용하는 추가적인 가정을 의미. 즉, 주어진 데이터만으로는 알 수 없는 새로운 상황에 대해 일반화(generalization)를 가능하게 만드는 전제들의 집합.
- 목적: 데이터로부터 학습한 것만으로는 일반화가 어렵기 때문에, 모델이 제대로 예측을 하도록 어떤 "추가적인 가정"을 도입하는 것.
- 예시: CNN(Convolutional Neural Network)은 '인접한 픽셀들이 관계가 있다'는 공간적 연관성의 inductive bias를, RNN(Recurrent Neural Network)은 '시퀀스의 순서가 중요하다'는 시간적 연관성의 inductive bias를 가지고 있음.
- 중요성: 적절한 inductive bias를 가진 모델은 한정된(제한된) 데이터에서도, 새로운 데이터에 대해 더 잘 예측(generalization)할 수 있음.
- 즉, inductive bias는 모델의 일반화 성능과 밀접하며, 모델마다 가지는 inductive bias가 다르기 때문에 특정 task에 맞는 모델을 선택할 때 중요한 기준이 됨

3.3 Inductive bias

Inductive bias in CNN

- In CNNs, **locality**, **two-dimensional neighborhood structure**, and **translation equivariance** are baked into each layer throughout the whole model
 - Locality**: Image의 feature는 특정 부분에 모여있음. 예를 들어 사람의 '코'라는 object는 (대충) image의 중앙 어딘가의 pixel들로 표현이 될 것. CNN 구조는 2차원에서 인접한 특정 부분에 대해 filter가 적용되기 때문에 자연스럽게 지역적인 feature를 뽑아내는 구조로 동작.
 - Translation equivariance**: Translation이란 image의 모든 pixel을 같은 방향으로 같은 거리만큼 이동하는 것을 말함(즉 image의 slide). translation equivariance는 input의 translation이 output에도 그대로 영향을 미친다는 의미. translation 후 함수를 적용하나, 함수를 적용하고 translation을 하나 결과가 같다고 할 수 있음.
 - two-dimensional neighborhood structure**: CNN에서는 필터가 2차원 이미지 위를 슬라이딩하면서 각 위치의 국소(local) 영역에 합성곱(conv) 연산을 수행해 특징 맵(feature map)을 만들어내고, 이 과정에서 이미지의 2차원 공간 구조가 자연스럽게 유지.



3.3 Inductive bias

Inductive bias in ViT(Transformer)

- Vision Transformer has much less image-specific inductive bias than CNNs.
- In ViT, only MLP layers are local and translationally equivariant, while the self-attention layers are global(MSA layer는 token 위치에 상관 없이 연산되기 때문에 local, translation equivariant한 특성이 없음). The two-dimensional neighborhood structure is used very sparingly: in the beginning of the model by cutting the image into patches and at fine-tuning time for adjusting the position embeddings for images of different resolution (as described below).
- Other than that, the position embeddings at initialization time carry no information about the 2D positions of the patches and all spatial relations between the patches have to be learned from scratch. (그 외에는 초기화 시의 위치 임베딩(position embeddings)이 패치들의 2D 위치에 대한 정보를 전혀 포함하지 않으며, 패치 간의 모든 공간적 관계는 학습 과정에서 처음부터 새로 학습해야 함)
- ImageNet과 같은 비교적 적은 양의 dataset으로 학습할 경우 ResNet 보다 성능이 조금 낮게 나왔는데 이는 CNN이 기본적으로 갖고 있는 "inductive bias (translation equivariance, locality)" 특성이 transformer에게는 없기 때문(by (self)attention). 하지만, large dataset(14M~300M)으로 학습하면 inductive bias이 없음에도 불구하고 좋은 성능을 내는 것을 확인.

3.4 [CLS] Token

■ [CLS] Token의 역할 in ViT

- [CLS] 토큰은 learnable token 으로서 입력 이미지 전체에 대한 요약 정보를 담기 위한 대표 토큰.
- 이 토큰을 통해 이미지 전체를 대표하는 특징(feature) 을 학습.
- 마지막에 이 [CLS] 토큰만을 classification head (MLP layer) 에 연결해서 이미지 분류 결과를 도출.
- Transformer는 본래 자연어 처리(NLP)용 구조였고, BERT 등에서 [CLS] 토큰은 문장의 의미를 요약(sequence전체의 의미를 함축)하는 용도로 사용. ViT는 이 아이디어를 차용해 이미지 전체를 대표하는 요약 정보로 [CLS] 토큰을 사용한 것.
- 만약 ViT를 detection이나 segmentation 같은 다른 task에 활용할 때는 [CLS] token을 쓰지 않거나 다르게 활용하는 구조도 많음. 예: DETR에서는 object query를 사용하고, [CLS] 는 쓰지 않음.

3.5 positional embedding

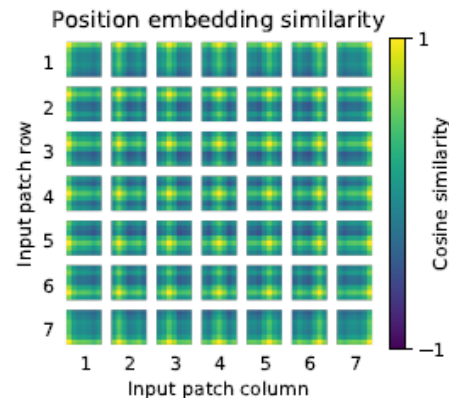
positional embedding in ViT

- learnable parameter
- patch embedding에 element wise로 더해지며 각 token(position) 별로 있음. 각 embedding을 N개 1D로 나열한 형태로 학습하였다. D/2 size로 N개의 X embedding, Y embedding을 각각 학습해 조합해서 사용하는 2D positional embedding도 사용해봤으나 큰 차이가 없었다.

- z_0 의 0은 0번 encoder block의 input이라는 의미

$$z_0 = [x_{\text{class}}; x_p^1 E; x_p^2 E; \dots; x_p^N E] + E_{\text{pos}}, \quad E \in \mathbb{R}^{(P^2 \cdot C) \times D}, E_{\text{pos}} \in \mathbb{R}^{(N+1) \times D} \quad (1)$$

- 아래 그림에서 i, j 번째 그림은 i, j 번째 positional embedding vector와, 모든 다른 positional embedding vector와의 cosine similarity를 나타낸다. 3, 3번째 그림을 보면 3,3 근처에서 제일 밝고, 같은 행, 열의 vector와 비슷한 유형을 보여준다. 즉, 학습된 positional embedding은 가깝거나 행, 열이 같은 positional embedding과 유사한 값을 갖는다.



3.6 Hybrid Architecture

Hybrid Architecture

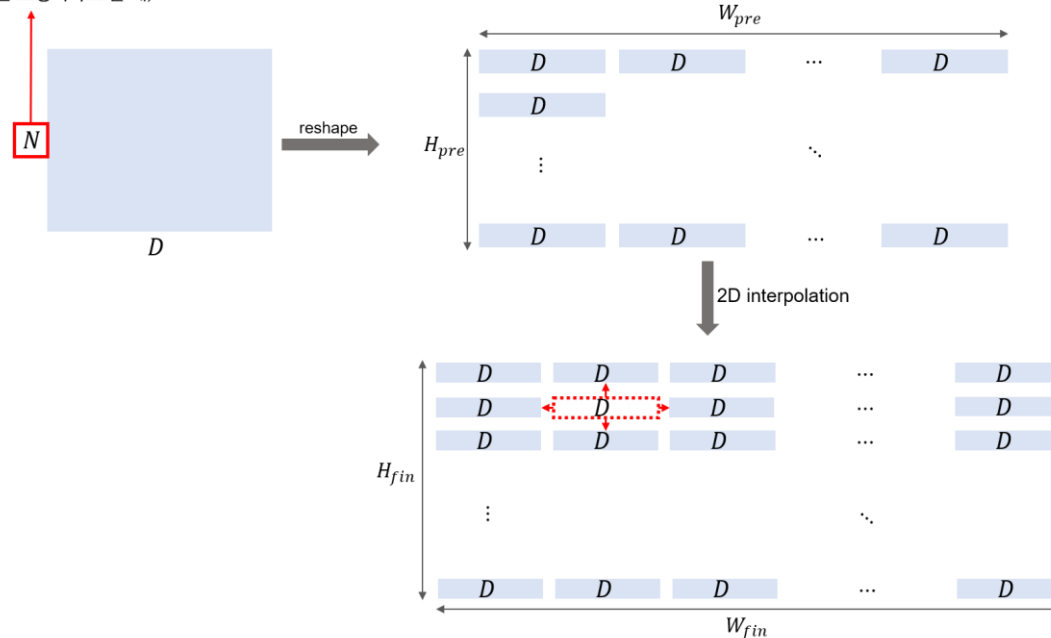
- CNN은 inductive bias로 (학습이 많이 안 됐을 경우) transformer에 비해 feature를 뽑는 능력이 좋기때문에, raw image를 patch로 자르기보다 CNN에서 뽑은 feature map을 cutting하여 patch로 만드는 hybrid 방법도 사용할 수 있다. 결론적으로 모델이 작을 때는 효과가 있었으나 model이 커짐에 따라 그 효과는 사라졌다.

3.7 Fine-tuning and Higher resolution

[CLS] Token의 역할 in ViT

- 일반적으로 많은 양의 dataset으로 pre-train을 한뒤 적은 양의 dataset으로 fine-tuning을 하게 된다. fine-tuning을 할 때는 pre-train에서 학습된 MLP head는 버리고 0으로 초기화 된 1개의 layer($D \times K$)만 넣어서 학습하는 게 고해상도 이미지에서 효과적이었다.
- ViT는 sequence의 길이에 상관없이 처리할 수 있다. 하지만 positional embedding은 특정한 길이의 sequence에 대해 학습된다. 고해상도 image의 경우 patch size가 동일해 sequence가 길어지게 되는데, 이때 짧은 sequence로 학습된 positional embedding을 그대로 사용하면 position의 정보가 별로 의미가 없게 된다. 따라서 학습된 sequence의 2D position 정보에 기반해 늘어난 sequence에 맞게 2D interpolation을 해서 positional embedding이 적절히 반영되게 한다.
- 이러한 해상도 조정과 패치 추출이 ViT 내에서 수동적으로 image-specific inductive bias를 추가해주는 부분.

입력 이미지의 해상도가 변화함에 따라
N의 값도 변화함.
(P는 고정이라고 할 때)



Chapter IV

ViT 실험 결과 및 한계

4.1 Comparison to State-Of-The-Art

Comparison to State-Of-The-Art

	Ours-JFT (ViT-H/14)	Ours-JFT (ViT-L/16)	Ours-I21k (ViT-L/16)	BiT-L (ResNet152x4)	Noisy Student (EfficientNet-L2)
ImageNet	88.55 ± 0.04	87.76 ± 0.03	85.30 ± 0.02	87.54 ± 0.02	88.4/88.5*
ImageNet ReaL	90.72 ± 0.05	90.54 ± 0.03	88.62 ± 0.05	90.54	90.55
CIFAR-10	99.50 ± 0.06	99.42 ± 0.03	99.15 ± 0.03	99.37 ± 0.06	—
CIFAR-100	94.55 ± 0.04	93.90 ± 0.05	93.25 ± 0.05	93.51 ± 0.08	—
Oxford-IIIT Pets	97.56 ± 0.03	97.32 ± 0.11	94.67 ± 0.15	96.62 ± 0.23	—
Oxford Flowers-102	99.68 ± 0.02	99.74 ± 0.00	99.61 ± 0.02	99.63 ± 0.03	—
VTAB (19 tasks)	77.63 ± 0.23	76.28 ± 0.46	72.72 ± 0.21	76.29 ± 1.70	—
TPUv3-core-days	2.5k	0.68k	0.23k	9.9k	12.3k

Table 2: Comparison with state of the art on popular image classification benchmarks. We report mean and standard deviation of the accuracies, averaged over three fine-tuning runs. Vision Transformer models pre-trained on the JFT-300M dataset outperform ResNet-based baselines on all datasets, while taking substantially less computational resources to pre-train. ViT pre-trained on the smaller public ImageNet-21k dataset performs well too. *Slightly improved 88.5% result reported in [Touvron et al. \(2020\)](#).

JFP로 학습된 ViT-H/14, ViT-L/16, ImageNet-21k로 학습된 ViT-L/16과 기존 SOTA인 BiT-L(JFP), Noisy Student(ImageNet, JFP)와의 비교.

ViT-L/16-JFT가 Bit-L보다 훨씬 계산량이 적으면서도 모든 task에서 성능이 우수. ViT-H는 여전히 BiT-L의 1/4밖에 계산량을 쓰지 않으면서도 더 우수한 성능을 보임.

ViT의 경우 모델 파라미터(모델 규모)가 증가하고, 데이터셋의 크기가 증가할 수록 지속적으로 성능이 증가. (no saturation)

4.2 Pre-training Data Requirements

CNN에 비해 적은 inductive bias를 갖고 있음에도 large dataset에서는 좋은 성능을 보임. dataset의 중요도를 알기 위해 두 가지 실험을 진행.

서로 다른 dataset을 사용해서 pretrain

- 가장 작은 ImageNet에 pre-train한 경우에는 ViT-L가 각각 patch size에 해당하는 ViT-B보다 성능이 안 나옴. ImageNet-21k에서는 비슷했고, 가장 큰 dataset인 JFT에서는 Large model의 성능이 더 좋아짐.
- 회색 영역은 다양한 사이즈의 BiT 결과의 영역인데 작은 dataset에서는 ViT에 비해 우수하지만 JFT에서는 ViT-H가 더 뛰어난 것을 확인할 수 있음.

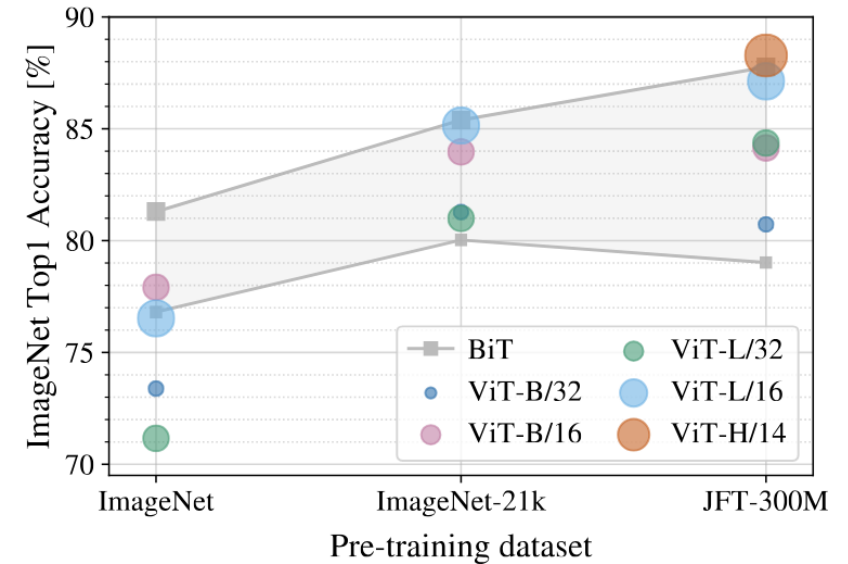


Figure 3: Transfer to ImageNet. While large ViT models perform worse than BiT ResNets (shaded area) when pre-trained on small datasets, they shine when pre-trained on larger datasets. Similarly, larger ViT variants overtake smaller ones as the dataset grows.

4.2 Pre-training Data Requirements

동일한 dataset에서 학습에 사용하는 양을 다르게

- JFT dataset에서 random sampling을 하여 학습 데이터의 양을 조절
 - 적은 dataset에서는 ViT-B/32, ViT-L/16은 overfit되어 성능이 좋지 않지만 30M부터는 ResNet50보다 좋아짐. 이는 CNN의 inductive bias가 smaller dataset에서 더 효과가 좋을 것이라는 직관에 대한 근거가 됨.
 - 반면 dataset이 커지면 inductive bias가 효과가 있을지라도 data로부터 그 특성을 충분히 찾아낼 수 있다는 것이 보임.
 - This result reinforces the *intuition* that the **convolutional inductive bias is useful for smaller datasets**, but **for large ones, learning the relevant patterns directly from data is sufficient**(충분), **even beneficial**(더 효과적).
 - 대규모 학습이 귀납적 편향(inductive bias)보다 더 중요하다는 것을 발견
 - 대규모 데이터로 충분히 학습하면, 설계상의 inductive bias 없이도 모델이 매우 좋은 성능을 낼 수 있다는 주장 (large scale training trumps inductive bias)

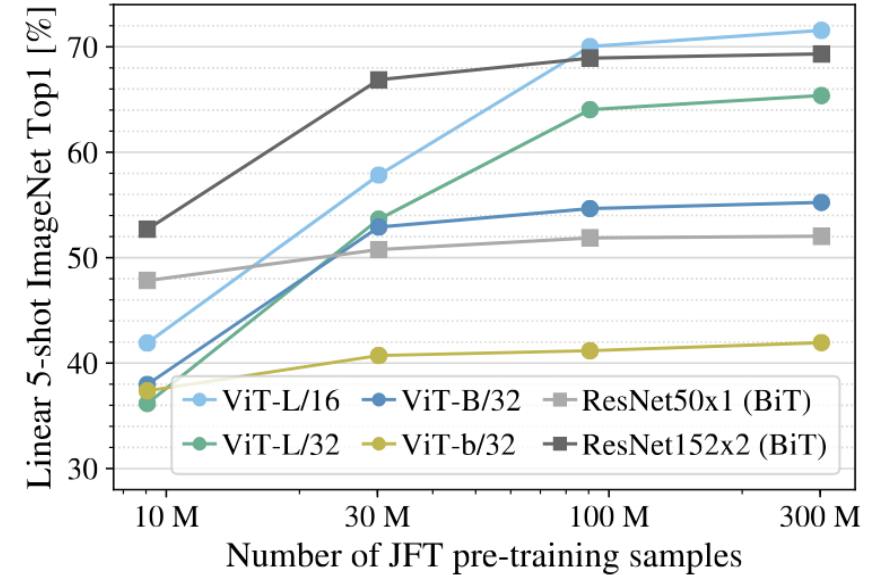


Figure 4: Linear few-shot evaluation on ImageNet versus pre-training size. ResNets perform better with smaller pre-training datasets but plateau sooner than ViT, which performs better with larger pre-training. ViT-b is ViT-B with all hidden dimensions halved.

4.3 Scaling Study

pre-training cost 대비 transfer learning (fine-tuning) 했을 때의 성능

- 같은 computational cost 대비 ViT가 ResNet보다 뛰어났으며 같은 성능을 위해서는 ResNet이 2~4배의 cost가 들었다.
- Hybrid의 경우 small dataset(low computation)에서 ViT보다 뛰어남을 보였으나 dataset이 커짐에 따라 그 차이는 없어졌다. CNN의 filter가 inductive bias로 local feature를 잘 뽑아내기 때문에 dataset의 크기에 상관 없이 도움이 될 것으로 예상할 수 있지만 실제로는 많은 data로부터 transformer의 inductive bias만으로도 feature를 잘 뽑아낼 수 있음을 나타낸다.
- ViT는 실험한 정도의 scale에서는 saturate하지 않았다. 즉, 더 큰 scale에서는 더 좋은 성능을 낼 가능성이 있다.

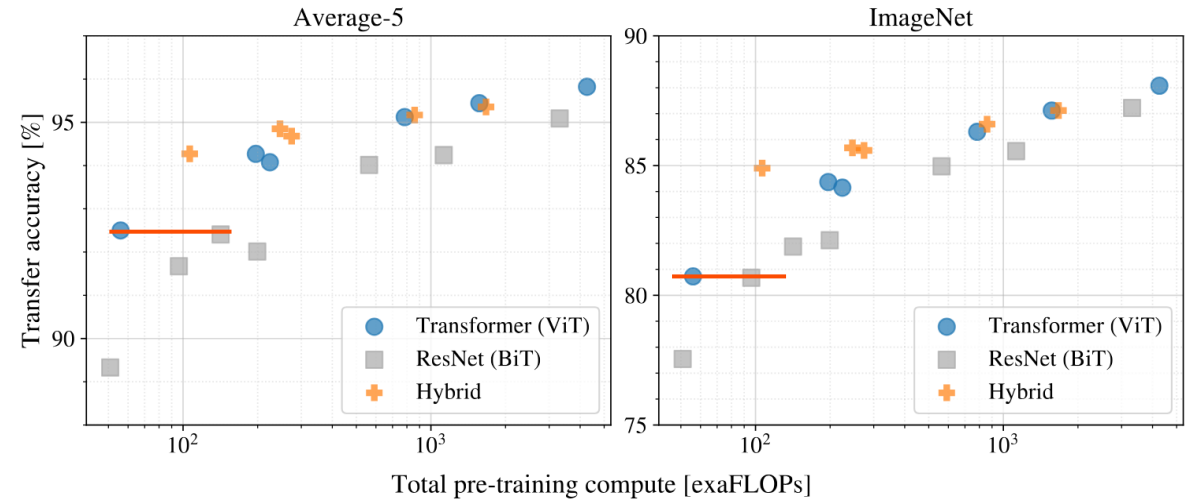


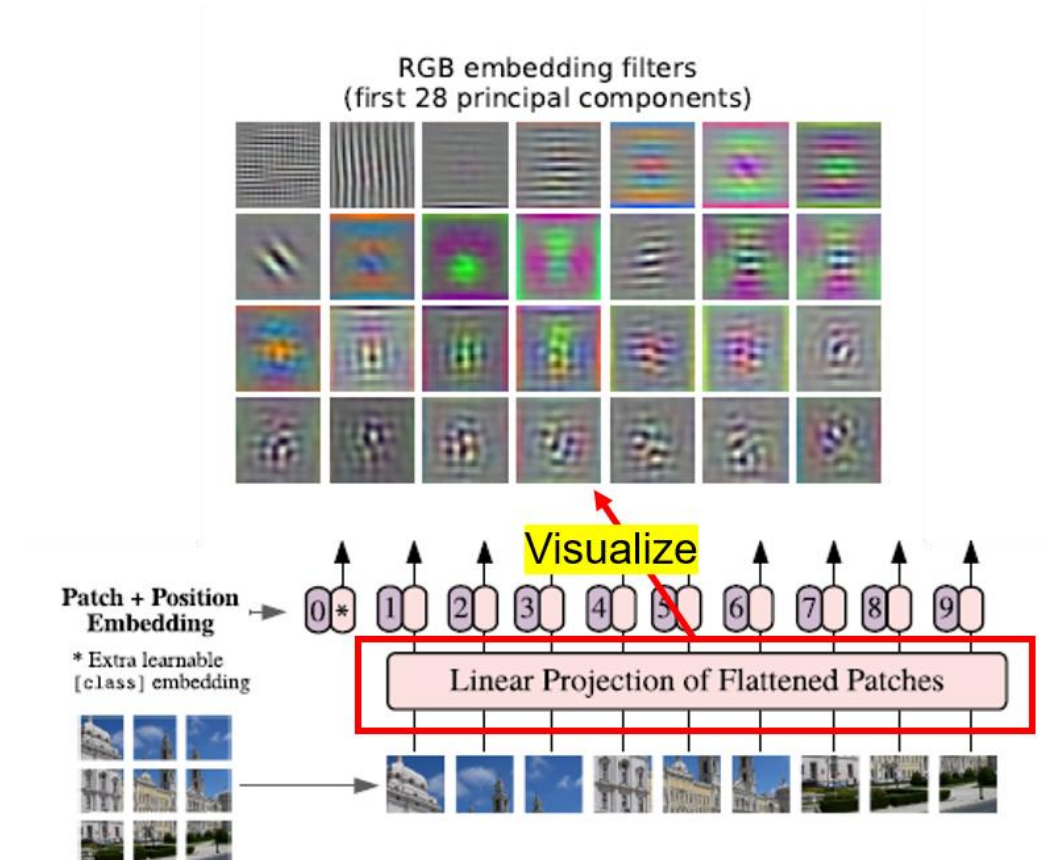
Figure 5: Performance versus pre-training compute for different architectures: Vision Transformers, ResNets, and hybrids. Vision Transformers generally outperform ResNets with the same computational budget. Hybrids improve upon pure Transformers for smaller model sizes, but the gap vanishes for larger models.

4.4 Inspection Vision Transformer

- ViT가 어떻게 image를 처리하는지 보기 위해 internal representation을 보았다.

Patch embedding layer (필터 시각화)

- 첫 번째 layer는 단순히 patch를 linear projection하여 patch embedding을 만든다. 오른쪽 그림은 이 embedding projection의 학습된 결과를 보여준다($P \times P \times C \times D$ projection은 D 개의 filter가 있다고 볼 수 있다.).
- CNN의 잘 학습된 앞쪽 레이어의 low level filter와 유사한 모습을 볼 수 있다. 즉, image low level feature(edge, color 등)를 잘 뽑아낼 수 있는 형태로 학습이 되어 있다는 의미이다. 2D geometrically feature를 뽑은 것이 아님에도 많은 dataset으로 학습하여 2D의 feature를 잘 뽑아낼 수 있게 되었다는게 포인트.
- 첫번째 linear projection 부분에서 주요 요소를 28개 선정하여 시각화함 (PCA를 통해서 D 개의 필터 중 28개를 선정한 것으로 추정).
- 기존의 $c \cdot p^2 \times D$ 크기의 가중치 행렬을 $D \times c \times p \times p$ 로 변환 후 시각화 했을 것으로 추정.



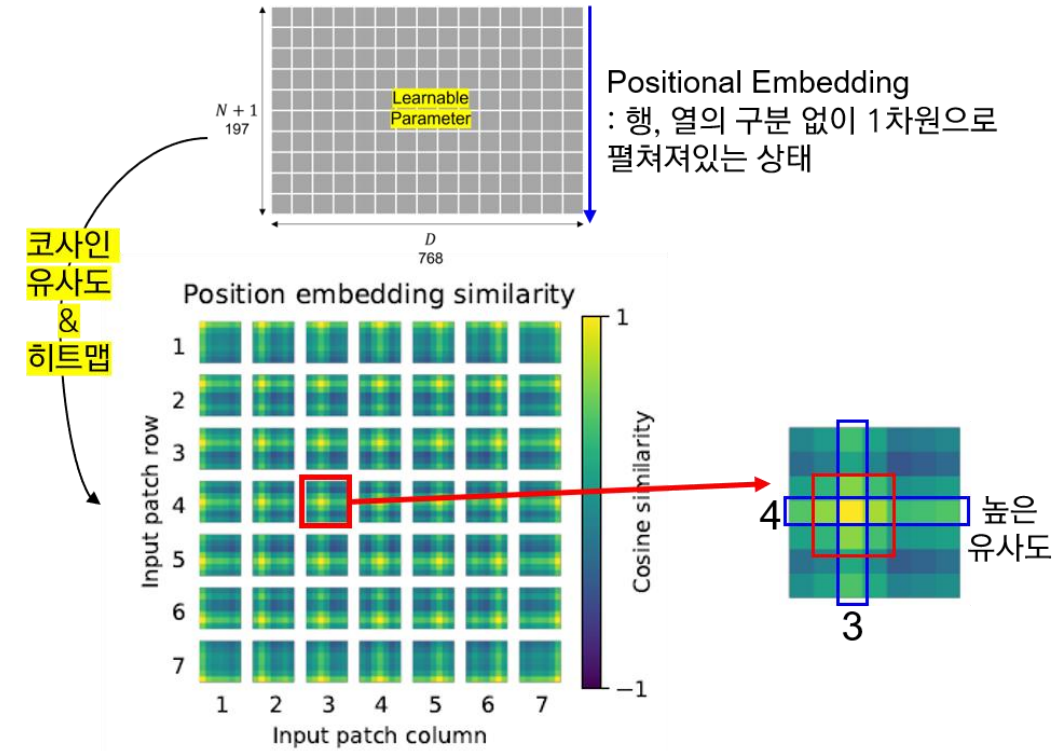
4.4 Inspection Vision Transformer

Positional embedding layer (위치 임베딩 유사도 분석)

- 행렬의 각 요소는 i 번째 패치 행, j 번째 패치 열에 대한 코사인 유사도 값의 히트맵으로 구성되어 있음.
- 위치 임베딩의 i 번째 열, j 번째 행 값이 이웃한 열, 행의 임베딩 값과 높은 유사도를 보임을 알 수 있음. 즉, **가까운 패치일 수록 높은 유사도**를 보여줌. 또한, 상대적으로 같은 열 & 같은 행의 위치 임베딩 값과 높은 유사도.
- 이는 ViT에서 1차원 임베딩을 사용했음에도 불구하고, 내부적으로 2차원의 이미지 위상을 잘 파악하고 있음을 알 수 있다. 즉, 1차원 위치 임베딩만으로도 2차원 이미지를 표현하기에 충분함을 시사함.
- 학습된 positional embedding이 2D topology를 잘 나타내기 때문에 manual하게 positional embedding을 사용하는 것보다 더 효율적이라는 의미.

Pos. Emb.	Default/Stem	Every Layer	Every Layer-Shared
No Pos. Emb.	0.61382	N/A	N/A
1-D Pos. Emb.	0.64206	0.63964	0.64292
2-D Pos. Emb.	0.64001	0.64046	0.64022
Rel. Pos. Emb.	0.64032	N/A	N/A

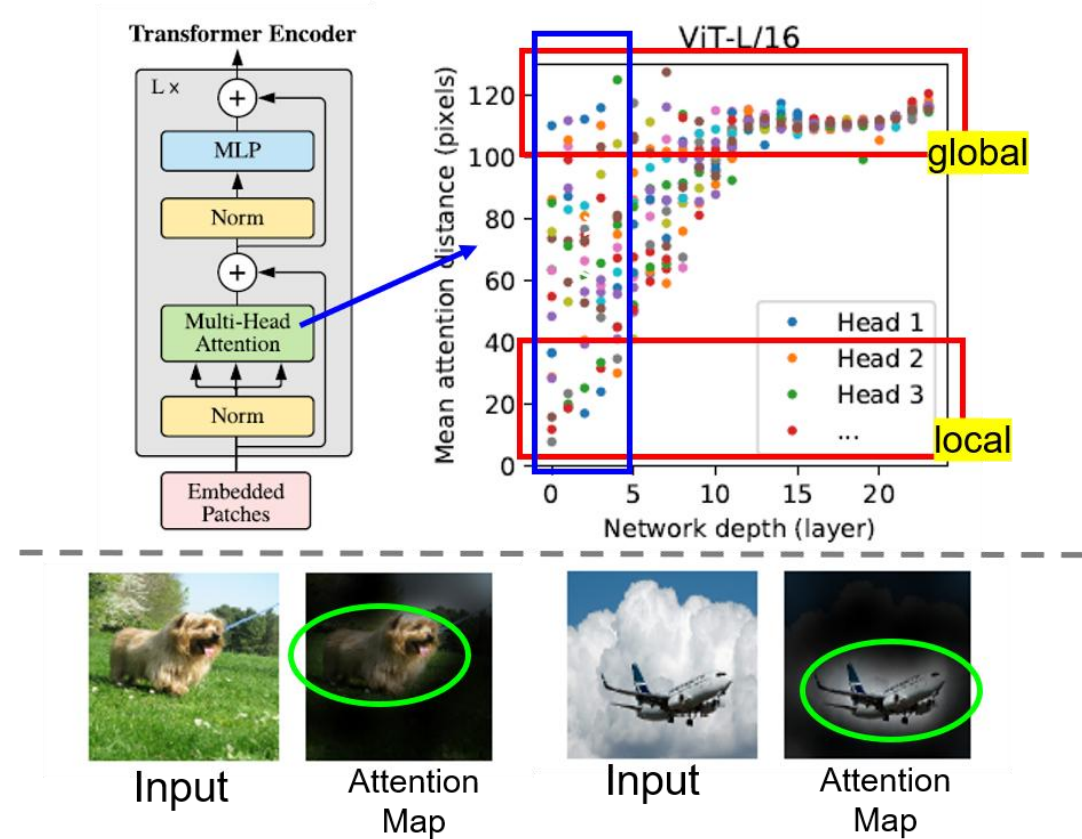
- 추가로 다양한 방식으로 positional embedding을 학습하도록 ablation을 해봤지만 차이는 미미했다. 이 이유는 patch 단위로 position이 들어가기 때문에 224 image 16 patch size라면 겨우 14x14 개의 position 정보만 알면 되기 때문에 인코딩 방식은 별로 안 중요하다고 함.



4.4 Inspection Vision Transformer

Attention Distance

- Attention Distance는 CNNs의 Receptive Field와 유사한 개념으로 볼 수 있음을 알고 넘어가자.
- Self-Attention은 네트워크가 가장 첫번째 레이어에서도 이미지 전체적인(global) 특성을 파악할 수 있도록 함.
이를 확인하기 위해 Attention weights를 바탕으로 Mean Attention Distance를 계산하여 한 픽셀에서 어느 정도 떨어져있는 픽셀까지 주목(attend)하는지 실험을 통해 보여줌.
- 위의 그림에서 알 수 있듯이 이미 첫번째 레이어에서부터 전역적 특성을 잡아내는 것이 가능함. 물론 지역적 특성 또한 잘 잡아내며, layer가 깊어질 수록 ViT 내부의 모든 헤드에서 대부분 전역적 특성을 잘 잡아냄을 알 수 있음. 최종적으로는 오른쪽 그림과 같이 분류작업과 관련 있는 부분을 attend하는 경향이 존재함.



Summary

I Summary

- Contribution
 - image recognition 분야에 Transformer를 직접적으로 적용한 첫 사례
 - 간단하면서 높은 확장성을 가진 Transformer 구조를 성공적으로 Image Recognition에 적용
 - 이미지에 특징된 inductive bias를 아키텍처에 주입하지 않음 (이미지 패치 추출 부분 제외)
 - 모델 학습 과정 전체적으로 이미지를 NLP에서 사용되던 것처럼 패치 시퀀스로 다룸
 - inductive bias가 약하기 때문에 보다 general-purpose의 architecture라고 할 수 있음. (one-for-all)
 - 큰 사이즈의 데이터셋에서 사전 학습한 후에 기존의 CNN 기반 baseline보다 더 좋은 성능. 추가적으로, 사전학습이 그리 어렵지 않다는 점.
 - 대용량의 dataset으로 pre-train한 뒤 상대적으로 작은 dataset(ImageNet, CIFAR, VTAB 등)으로 transfer(fine-tuning) 했을 때 적은 계산량을 요구함과 동시에 성능이 좋았음.
 - 이미지로부터 특징(feature)을 추출하는 Visual Encoder로써 많이 사용되고 있음 → ViT가 백본(Backbone) 역할로 쓰인다는 의미.
- Challenges
 - 본 연구에서는 classification task에만 적용하였기에 segmentation, detection 등의 다른 computer vision task에도 적용이 성공적으로 되는지는 아직 정확히 알 수 없음.
 - 사전학습 시, self-supervised learning 방법론을 성공적으로 적용시킬 방안을 탐색해야 함.
 - self-supervised learning 방식으로 사전학습을 해서 나쁘지 않은 결과를 얻었지만 large-scale supervised pre-training과는 큰 차이가 있었음.
 - 모델 구조 개선을 통한 모델 성능 향상의 여지가 존재함.
 - 중간 사이즈 혹은 그보다 작은 데이터 셋을 학습시킬 경우, 별도의 강력한 정규화 없이 기존 ResNets 대비 훌륭한 성능을 보이진 못했음
 - CNN에 비해 Inductive Bias가 부족하여, 많은 양의 데이터가 없이 일반화가 제대로 이뤄지지 않았을 것으로 추정
 - 하지만 큰 사이즈의 Dataset을 학습 시킬 경우, 위 한계를 극복 가능함

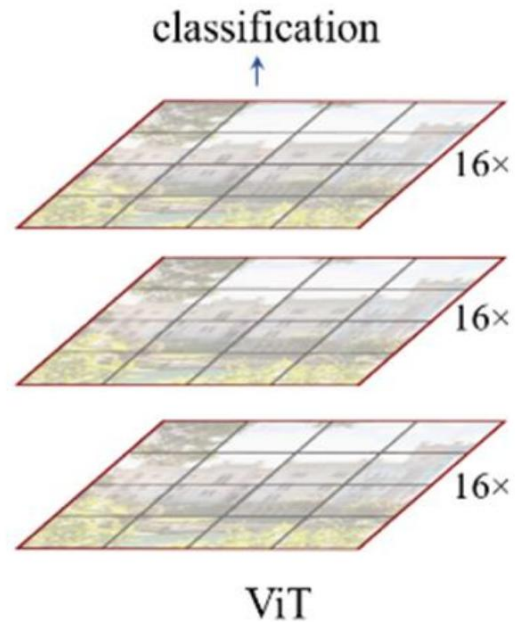
Chapter V

SwinT Intro

5.0 Purpose

- Previous Problem
 - 기존 ViT 모델은 분류 문제를 풀기 위한 모델로 제안
 - 텍스트와 달리 이미지를 위한 특성이 ViT에 없음
 - Token의 수가 증가함에 따라 연산량이 quadratic하게 증가
- Purpose
 - 다양한 목적에 Backbone으로 사용될 수 있는 모델 제안
 - Transformer 구조에 이미지의 특성을 반영할 수 있는 방법 제안
 - 기존 ViT 모델보다 더 적은 연산량을 갖는 방법 제안

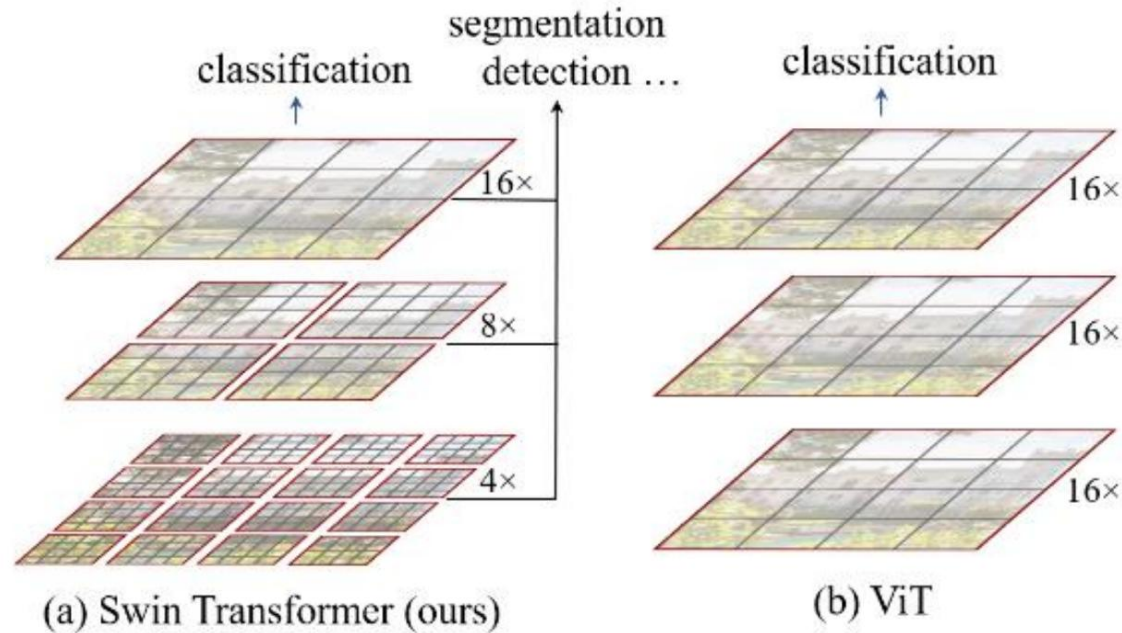
5.1 Current deep ViT



What is wrong with this?

Challenges in adapting Transformer from language to vision arise from differences between the two domains, such as **large variations in the scale of visual entities** and the **high resolution of pixels in images compared to words in text**. To address these differences, we propose a hierarchical Transformer whose representation is computed with Shifted windows.

5.2 Hierarchical Vision Transformer using Shifted Windows



Ideas:

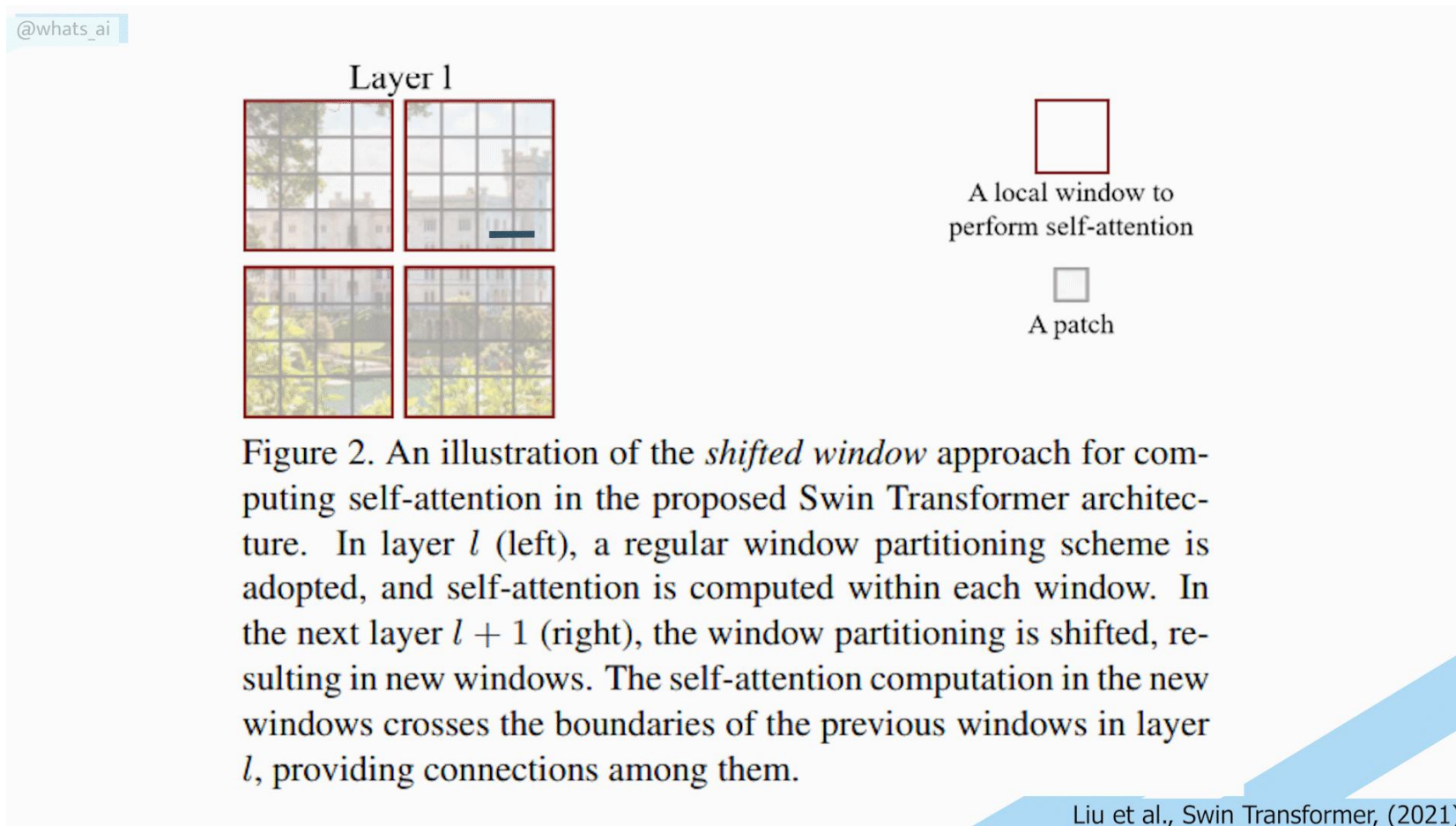
- Use smaller patches (4x4x3)
- Project them to lower dimension (4)
- Merge tokens at deeper levels
- Full attention => Window attention
 - => Shifted window attention

Swin Transformer: Hierarchical Vision Transformer using Shifted Windows

Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, Baining Guo

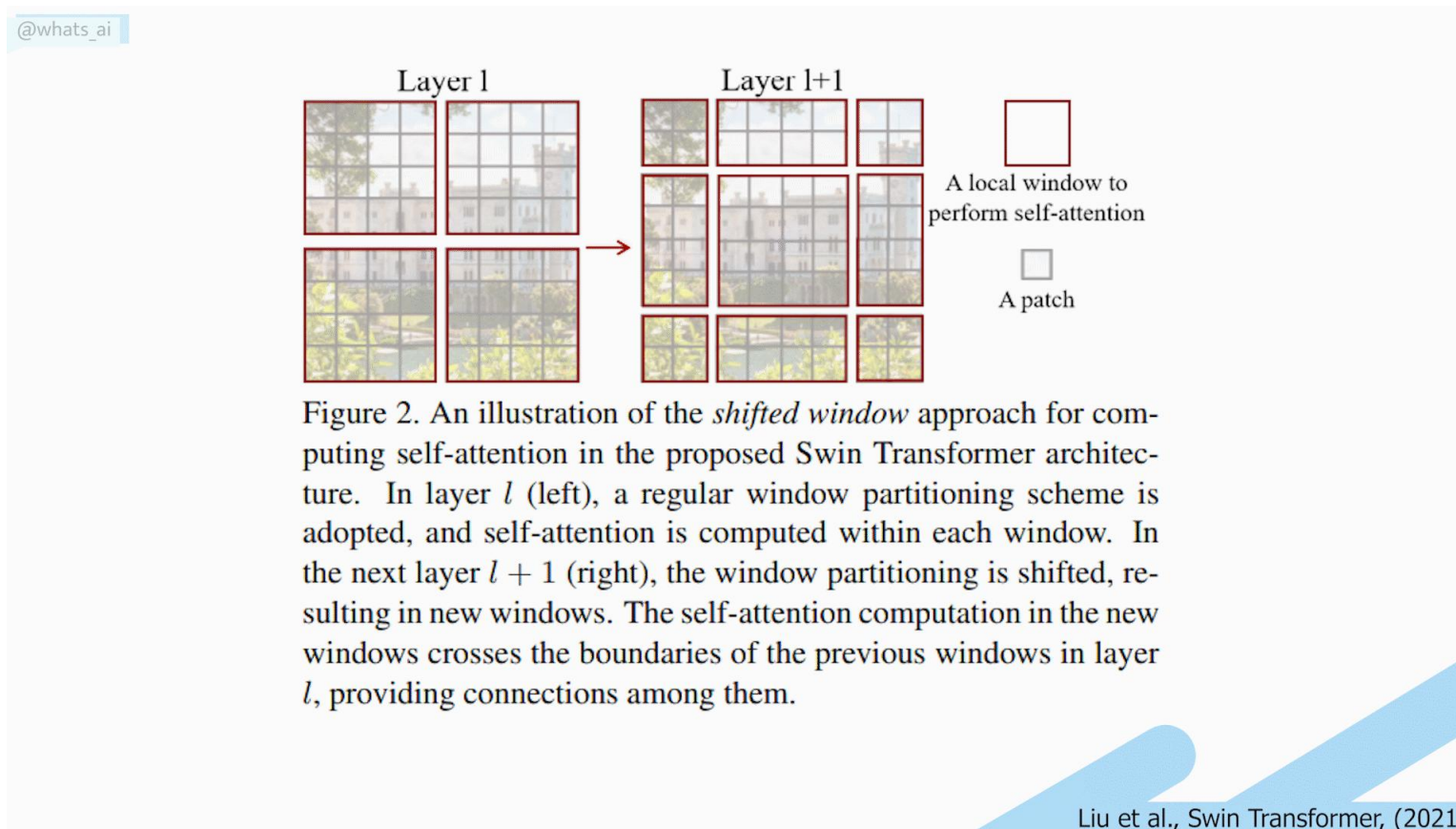
5.3 Shifted Window Attention

Self-attention applied on windows



5.4 Shifting window long-range relation problem

- Swin Transformer의 Shifted Window 기법은 장거리 픽셀 간 관계를 형성할 수 있도록 해줌. 하지만 아쉽게도 이러한 장거리 관계는 서로 인접한 윈도우 간에만 발생할 수 있음.
- 즉, 진짜로 멀리 떨어진 영역 간의 관계는 여전히 포착되지 못한다는 한계가 있음. 이는 컴퓨터 비전 분야에서 Transformer 아키텍처가 여전히 개선의 여지가 있다는 것을 보여줌.
- --> 근데 이미지는 text처럼 문맥이 크게 중요하지 않은 것 같은데 굳이 멀리 떨어진 영역 간의 관계를 attention을 통해 계산해야 할까? (개인적인 의견)



Chapter VI

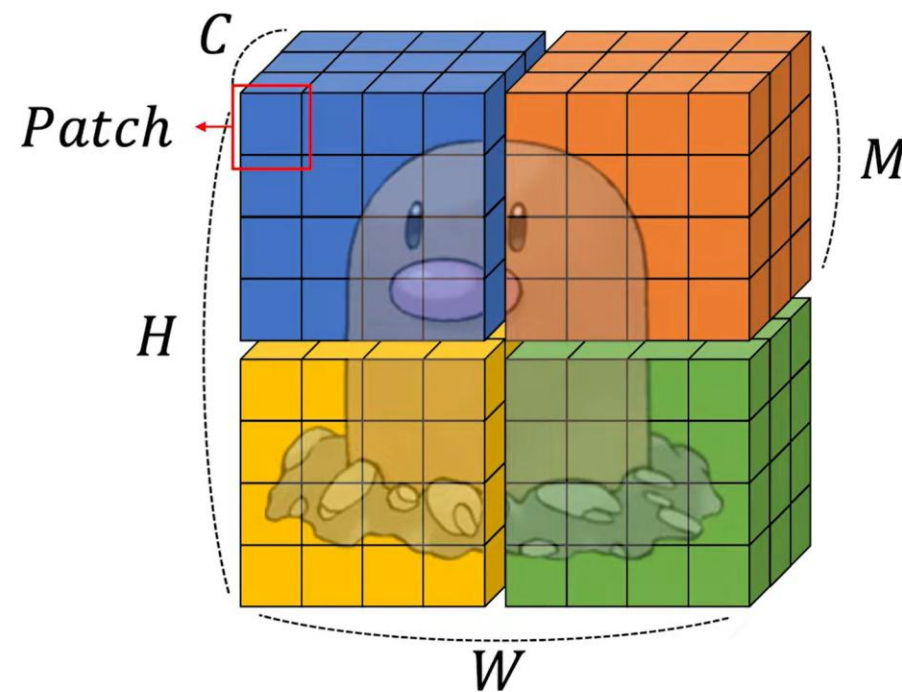
Architecture of SwinT

6.1 Notation

Notation

- H : 이미지 세로 길이 (Height)
- W : 이미지 가로 길이 (Width)
- C : 이미지 채널 수 (Channel)
- M : local window 크기
- n : local window 개수
- Patch: 이미지 내에서 분할된 이미지(token)
- P_h, P_w : patch 크기
- $N (= N_h \times N_w)$: patch(token) 개수 (세로, 가로 개수)
- B : 배치 사이즈

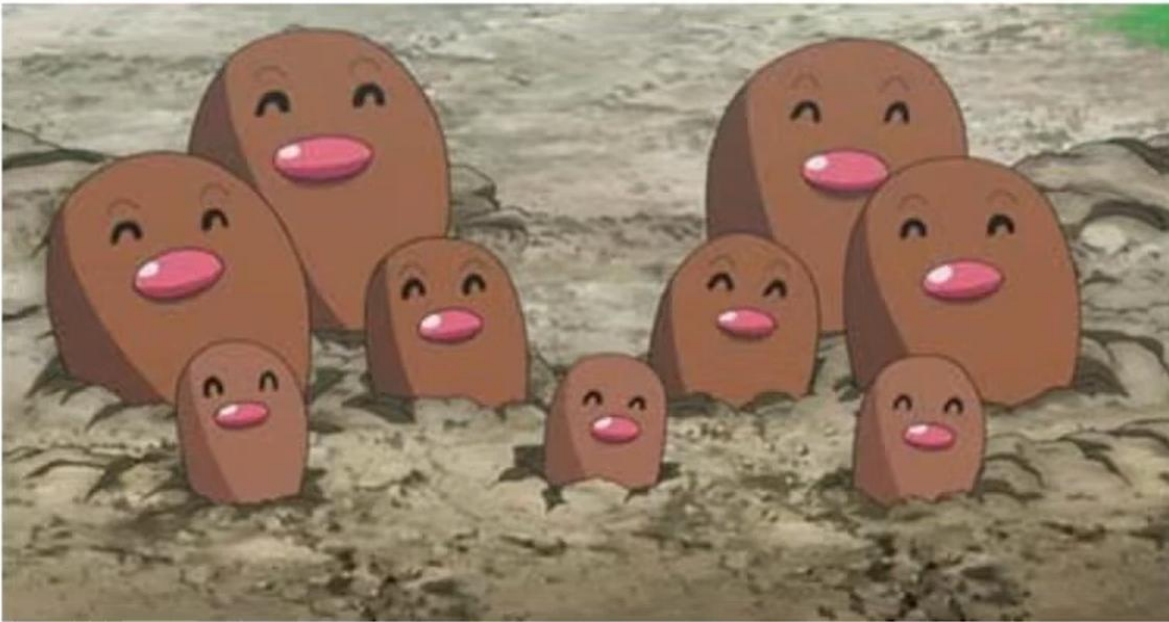
- $H \times W \times C : 32 \times 32 \times 3$
- $M: 4$
- $n: 4$
- $P_h, P_w: 4$
- $N: 64$



6.2 Solution

텍스트와 다른 이미지의 특성

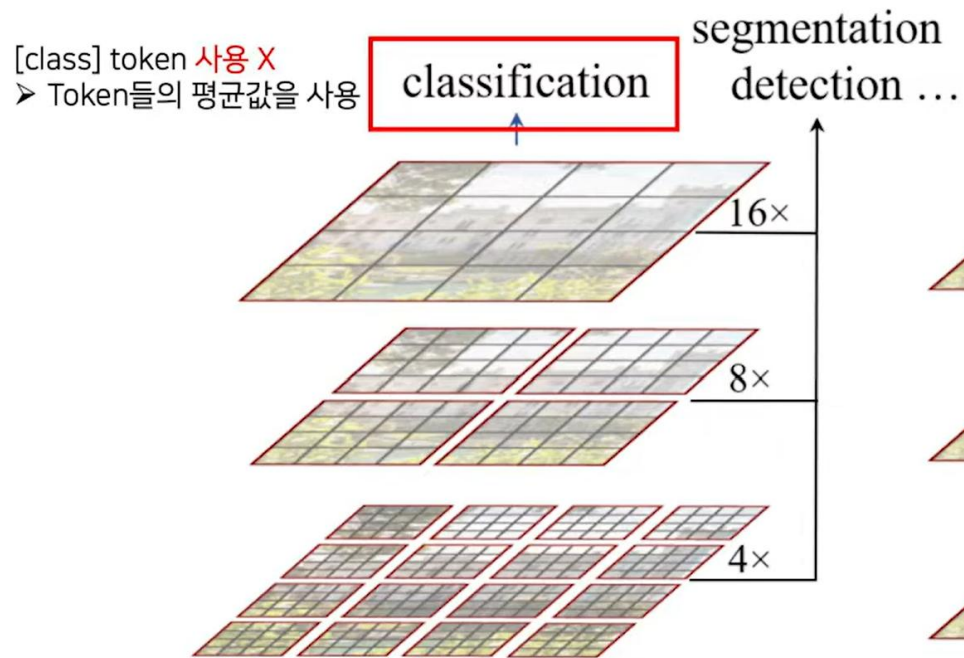
- 해상도 (resolution)
- 물체의 크기 (the scale of visual entities)



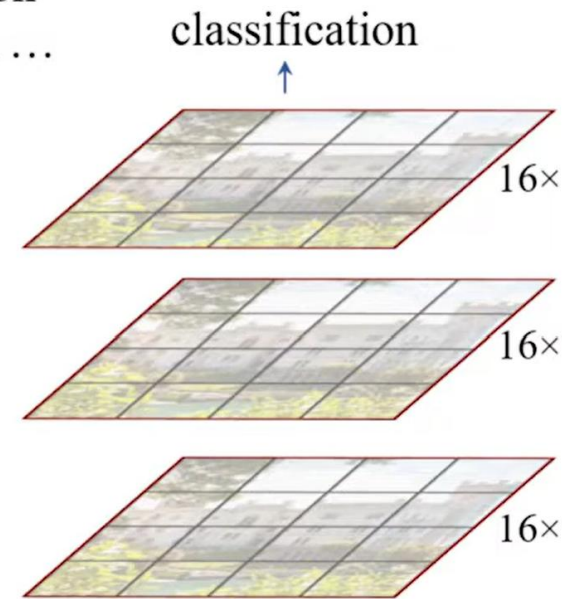
6.2 Solution

제안 방법

- Local window를 모델에 적용
- 계층마다 다른 해상도
 - Segmentation or detection task를 수행하기에 적합한 백본 형태의 모델
- 최종적으로는 윈도우 == 전체 이미지 사이즈
 - 이미지의 모든 패치를 볼 수 있음



(a) Swin Transformer (ours)



(b) ViT

6.2 Solution

제안 방법

- Local window를 모델에 적용

✓ 기존 ViT보다 더 적은 complexity로 학습

$$\Omega(\text{MSA}) = 4hwC^2 + 2(hw)^2C$$

$$\Omega(\text{W-MSA}) = 4hwC^2 + 2M^2hwC$$

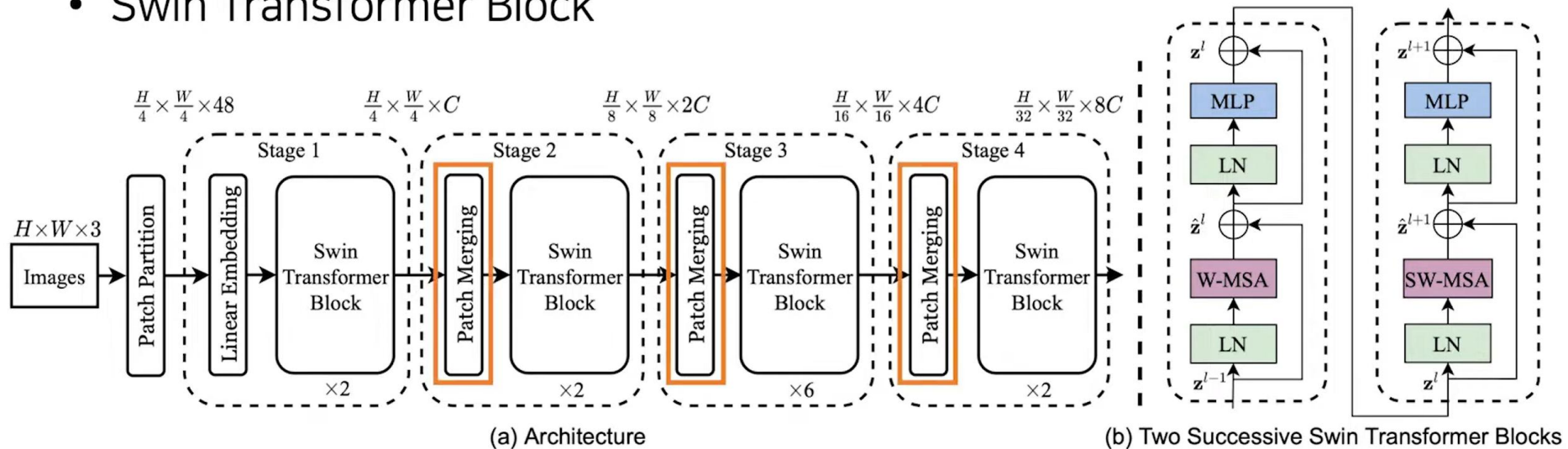
- h, w : 세로, 가로 patch 개수
- M : local window 사이즈

6.3 Method

모델 구조

- Patch Merging -> 해상도 줄여줌
- Swin Transformer Block

- Patch Merging
- Swin Transformer Block



W-MSA (Window Multi-head Self Attention)

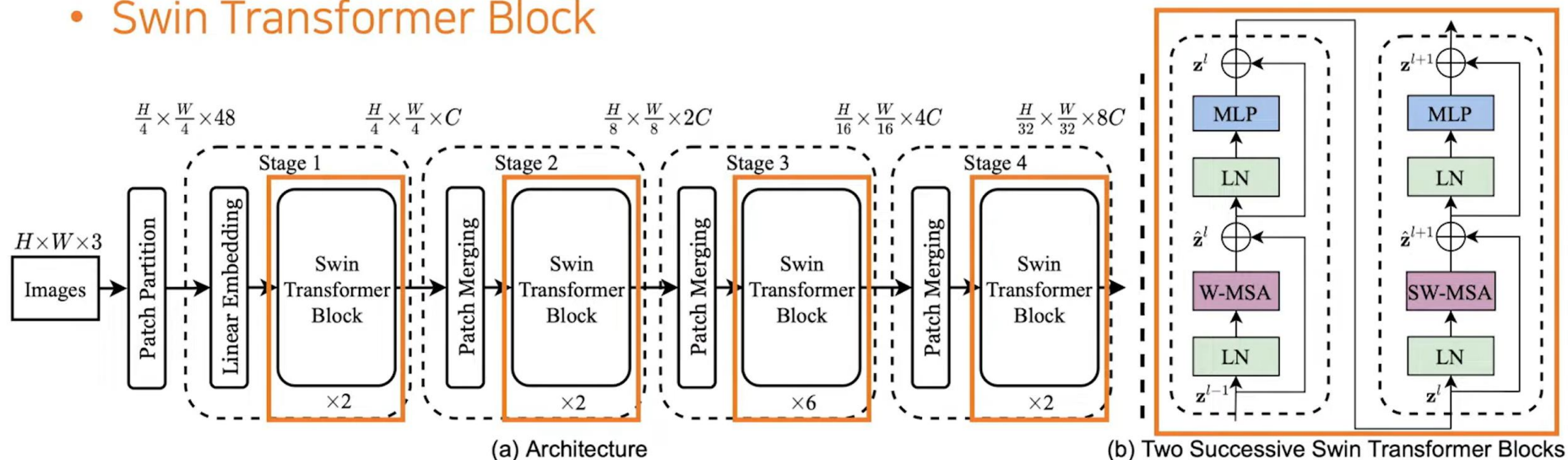
SW-MSA (Shifted Window Multi-head self Attention)

6.3 Method

모델 구조

- Patch Merging
- Swin Transformer Block

- Patch Merging
- Swin Transformer Block

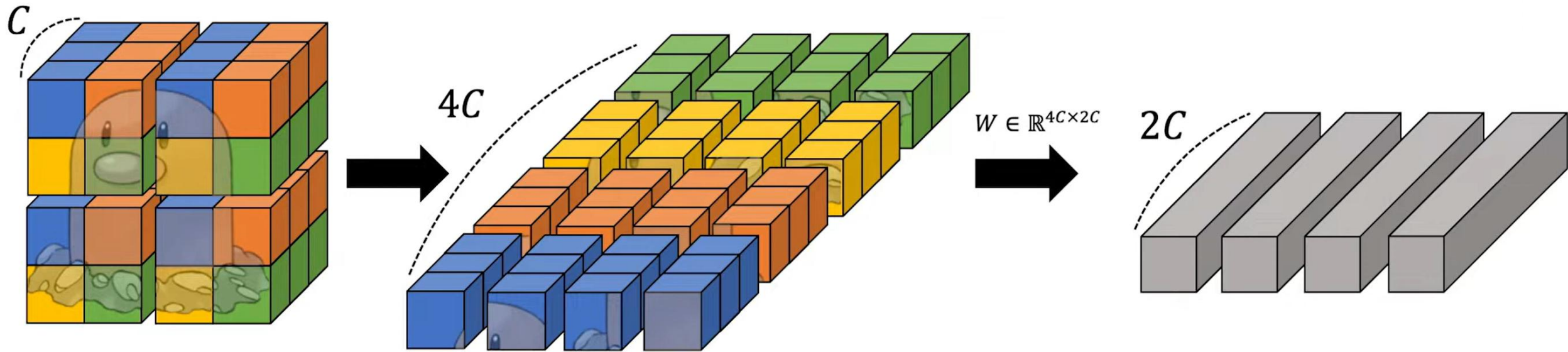


W-MSA (Window Multi-head Self Attention)
 SW-MSA (Shifted Window Multi-head self Attention)

6.3 Method

Patch Merging

- 주변의 2x2 패치를 가져다가 하나의 차원으로 축소
- Linear projection 으로 2c차원으로 reduction.
- 논문에 구체적인 언급은 없지만 스테이지를 거듭하면서 patch merging을 통해 reduction없이 그냥 concat 만 하면 C차원이 매우 커져서 이걸 방지하기 위해 사용한 것이지 않을까 + concat 된 feature들을 섞어주는 역할도



6.3 Method

Swin Transformer Block

Swin Transformer Block

- W-MSA : Local Window 안에서 self attention
- SW-MSA : Local Window 간의 self attention

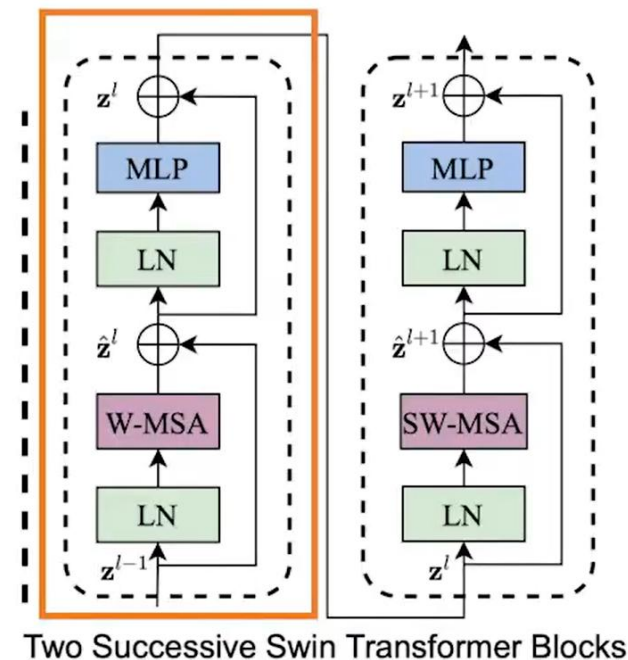
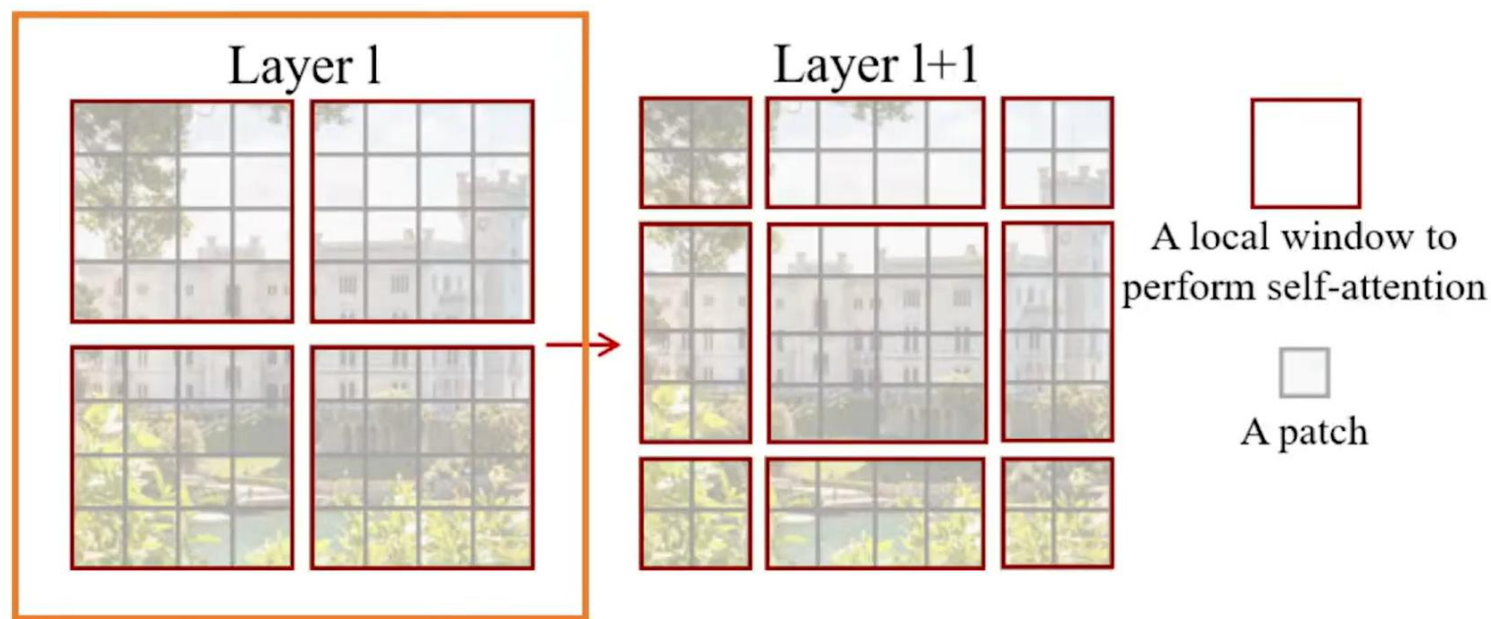
Details

- Efficient batch Computation
- Relative Position Bias
- Cyclic Shift & Attention Mask

6.3 Method

Swin Transformer Block

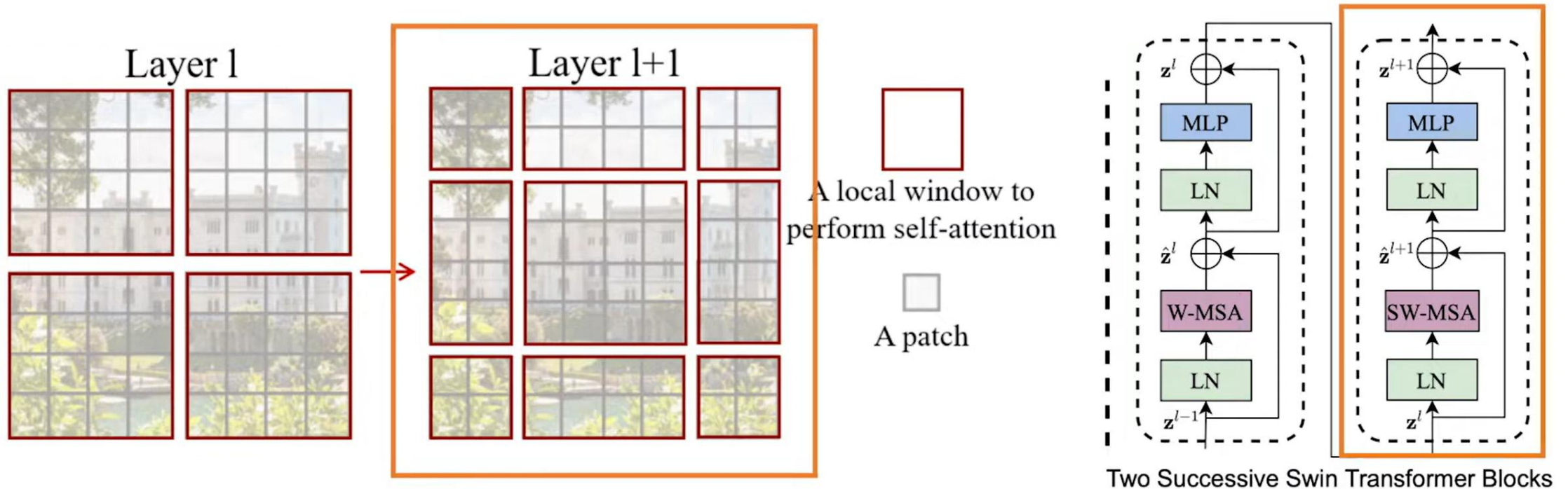
- **W-MSA** : Local Window 안에서 self attention
- **SW-MSA** : Local Window 간의 self attention



6.3 Method

Swin Transformer Block

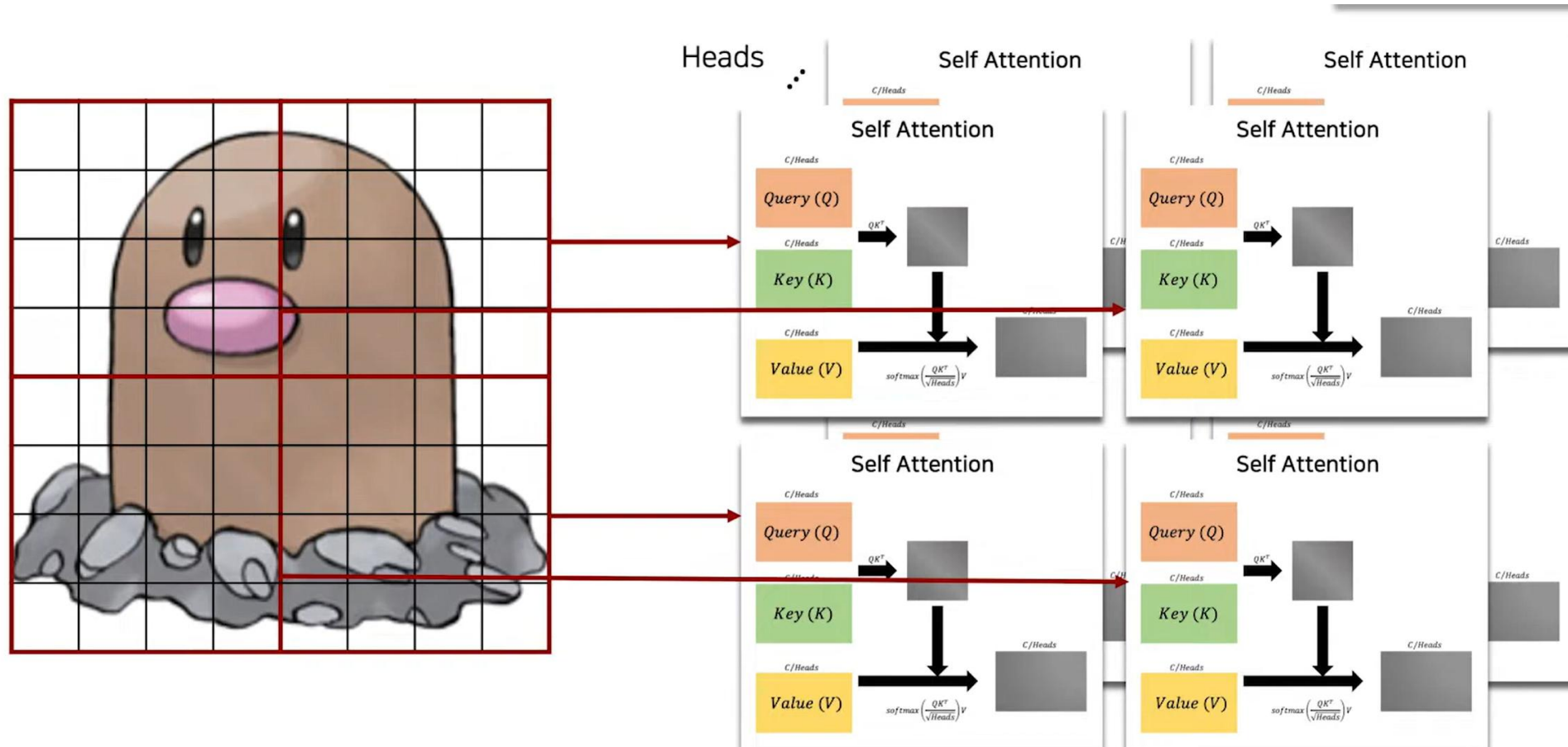
- **W-MSA** : Local Window 안에서 self attention
- **SW-MSA** : Local Window 간의 self attention



6.3 Method

Swin Transformer Block : W-MSA

- 전체 이미지에 대해 할 때보다 4번 반복해서 수행해야 하나? -> 더 효율적으로 하자 -> batch computation

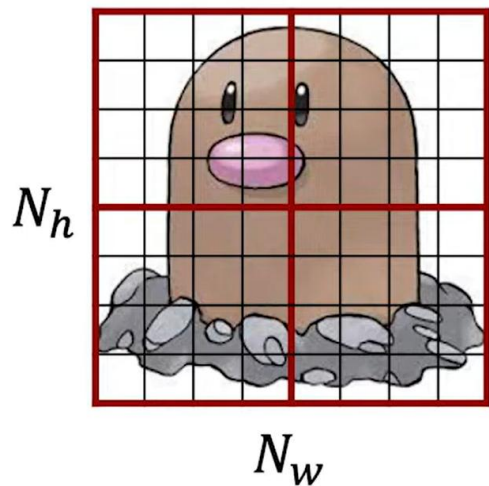


6.3 Method

Swin Transformer Block : Efficient Batch computation

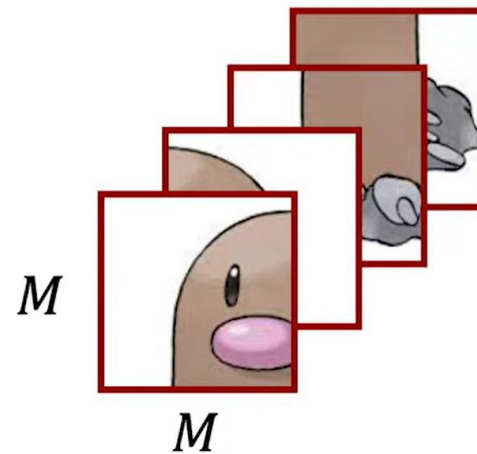
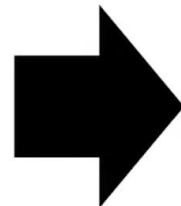
- n 을 배치와 같은 차원으로 합쳐서 병렬로 한 번에 연산

$$x \in \mathbb{R}^{B \times N_h \times N_w \times C}$$



$$x \in \mathbb{R}^{1 \times 8 \times 8 \times 3}$$

$$x \in \mathbb{R}^{nB \times M \times M \times C}, n = \frac{N_h}{M} + \frac{N_w}{M}$$



$$x \in \mathbb{R}^{4 \times 4 \times 4 \times 3}$$

$$M = 4, n = 4$$

6.3 Method

Swin Transformer Block : Relative Position Bias

- Vision Transformer(ViT)는 전체 이미지에 global attention을 사용하여 positional encoding이 중요
- 하지만 Swin Transformer는 작은 윈도우 내에서만 attention을 수행하기 때문에, 절대 위치보다는 상대적인 위치 차이가 더 중요
- local window 기반 구조와 더 잘 어울리는 relative positional bias
 - 두 축마다 Relative Position의 범위 $[-M + 1, M - 1]$
 - Bias Index Matrix ($\hat{B}$) $\in \mathbb{R}^{(2M-1) \times (2M-1)}$
 - B 는 $\hat{B}$ 의 값을 사용

$$\text{Attention}(Q, K, V) = \text{SoftMax}\left(QK^T / \sqrt{d} + \boxed{B}\right)V$$

6.3 Method

Swin Transformer Block : Relative Position Bias

Window size (M) = 3

x axis

1	2	3
4	5	6
7	8	9

x axis matrix

	1	2	3	4	5	6	7	8	9
1	0	0	0	-1	-1	-1	-2	-2	-2
2	0	0	0	-1	-1	-1	-2	-2	-2
3	0	0	0	-1	-1	-1	-2	-2	-2
4	1	1	1	0	0	0	-1	-1	-1
5	1	1	1	0	0	0	-1	-1	-1
6	1	1	1	0	0	0	-1	-1	-1
7	2	2	2	1	1	1	0	0	0
8	2	2	2	1	1	1	0	0	0
9	2	2	2	1	1	1	0	0	0

y axis

1	2	3
4	5	6
7	8	9

y axis matrix

	1	2	3	4	5	6	7	8	9
1	0	-1	-2	0	-1	-2	0	-1	-2
2	1	0	-1	1	0	-1	1	0	-1
3	2	1	0	2	1	0	2	1	0
4	0	-1	-2	0	-1	-2	0	-1	-2
5	1	0	-1	1	0	-1	1	0	-1
6	2	1	0	2	1	0	2	1	0
7	0	-1	-2	0	-1	-2	0	-1	-2
8	1	0	-1	1	0	-1	1	0	-1
9	2	1	0	2	1	0	2	1	0

y axis

1	2	3
4	5	6
7	8	9

y axis matrix

	1	2	3	4	5	6	7	8	9
1	0	-1	-2	0	-1	-2	0	-1	-2
2	1	0	-1	1	0	-1	1	0	-1
3	2	1	0	2	1	0	2	1	0
4	0	-1	-2	0	-1	-2	0	-1	-2
5	1	0	-1	1	0	-1	1	0	-1
6	2	1	0	2	1	0	2	1	0
7	0	-1	-2	0	-1	-2	0	-1	-2
8	1	0	-1	1	0	-1	1	0	-1
9	2	1	0	2	1	0	2	1	0

6.3 Method

Swin Transformer Block : Relative Position Bias

Index로 나타내기 위해 값의 범위가 0부터 시작되도록 변환

```
# Step 1
x_axis_matrix += window_size - 1
y_axis_matrix += window_size - 1

# Step 2
x_axis_matrix *= 2 * window_size - 1
relative_position_M = x_axis_matrix + y_axis_matrix
```

Relative Position index

12	11	10	7	6	5	2	1	0
13	12	11	8	7	6	3	2	1
14	13	12	9	8	7	4	3	2
17	16	15	12	11	10	7	6	5
18	17	16	13	12	11	8	7	6
19	18	17	14	13	12	9	8	7
22	21	20	17	16	15	12	11	10
23	22	21	18	17	16	13	12	11
24	23	22	19	18	17	14	13	12

6.3 Method

Swin Transformer Block : Relative Position Bias

```
# Step 1
x_axis_matrix += window_size - 1
y_axis_matrix += window_size - 1

# Step 2
x_axis_matrix *= 2 * window_size - 1
relative_position_M = x_axis_matrix + y_axis_matrix
```

Relative Position index

12	11	10	7	6	5	2	1	0
13	12	11	8	7	6	3	2	1
14	13	12	9	8	7	4	3	2
17	16	15	12	11	10	7	6	5
18	17	16	13	12	11	8	7	6
19	18	17	14	13	12	9	8	7
22	21	20	17	16	15	12	11	10
23	22	21	18	17	16	13	12	11
24	23	22	19	18	17	14	13	12

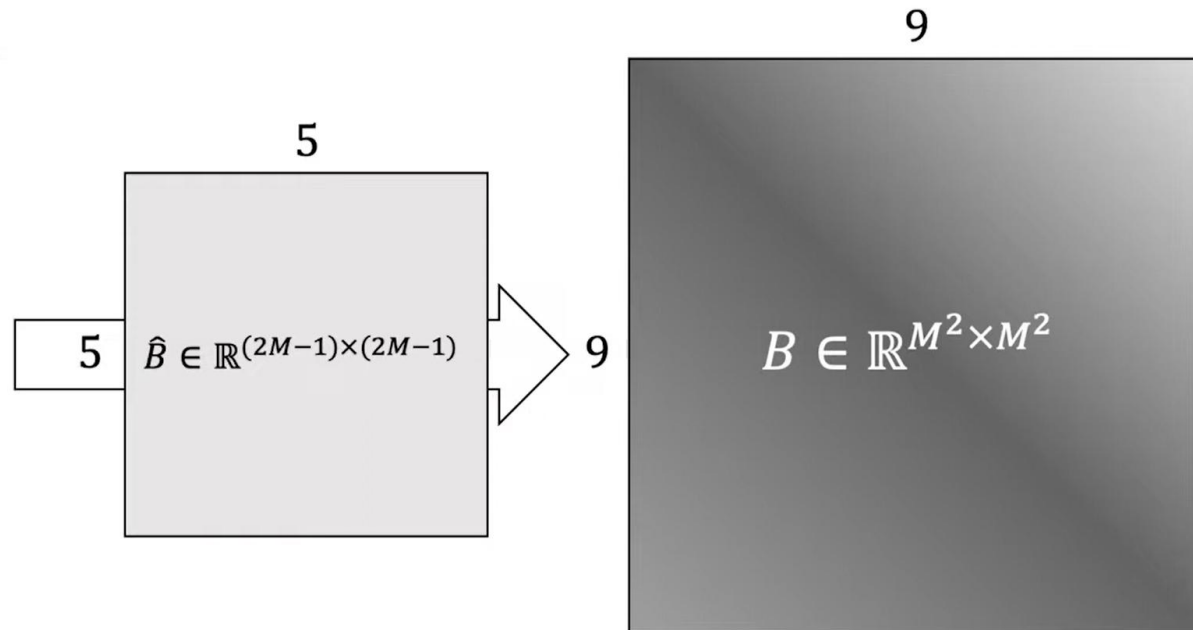
6.3 Method

Swin Transformer Block : Relative Position Bias

Window size (M) = 3

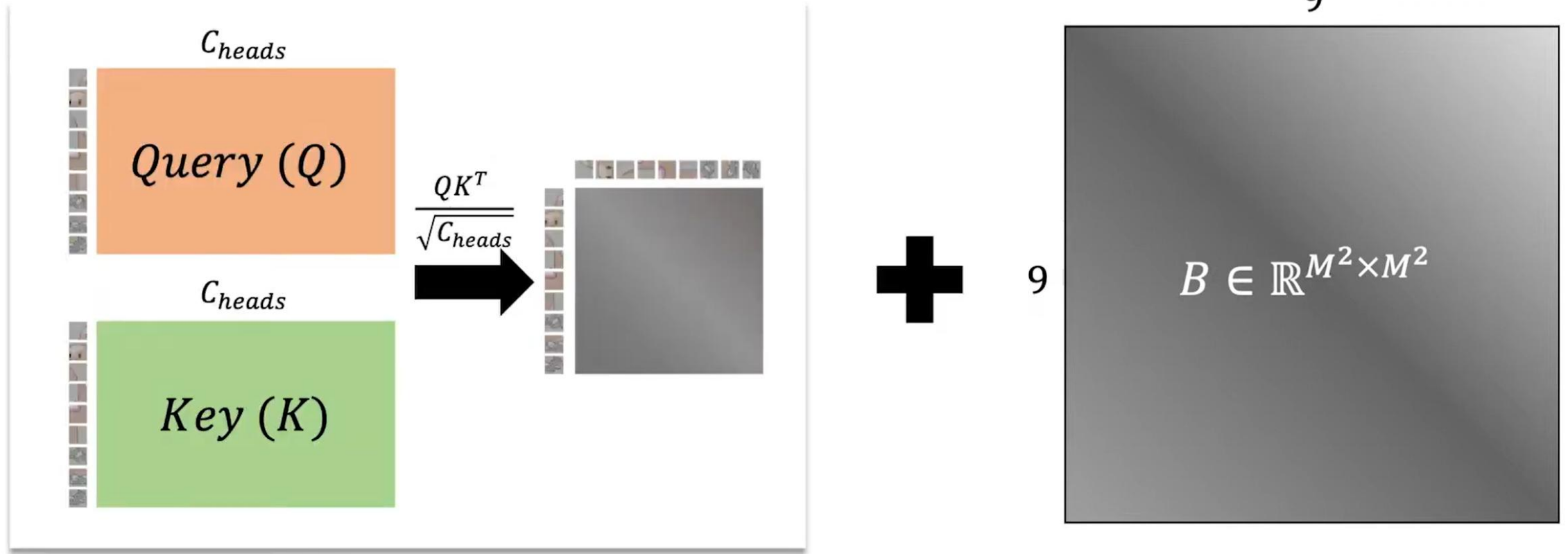
Relative Position index

12	11	10	7	6	5	2	1	0
13	12	11	8	7	6	3	2	1
14	13	12	9	8	7	4	3	2
17	16	15	12	11	10	7	6	5
18	17	16	13	12	11	8	7	6
19	18	17	14	13	12	9	8	7
22	21	20	17	16	15	12	11	10
23	22	21	18	17	16	13	12	11
24	23	22	19	18	17	14	13	12



6.3 Method

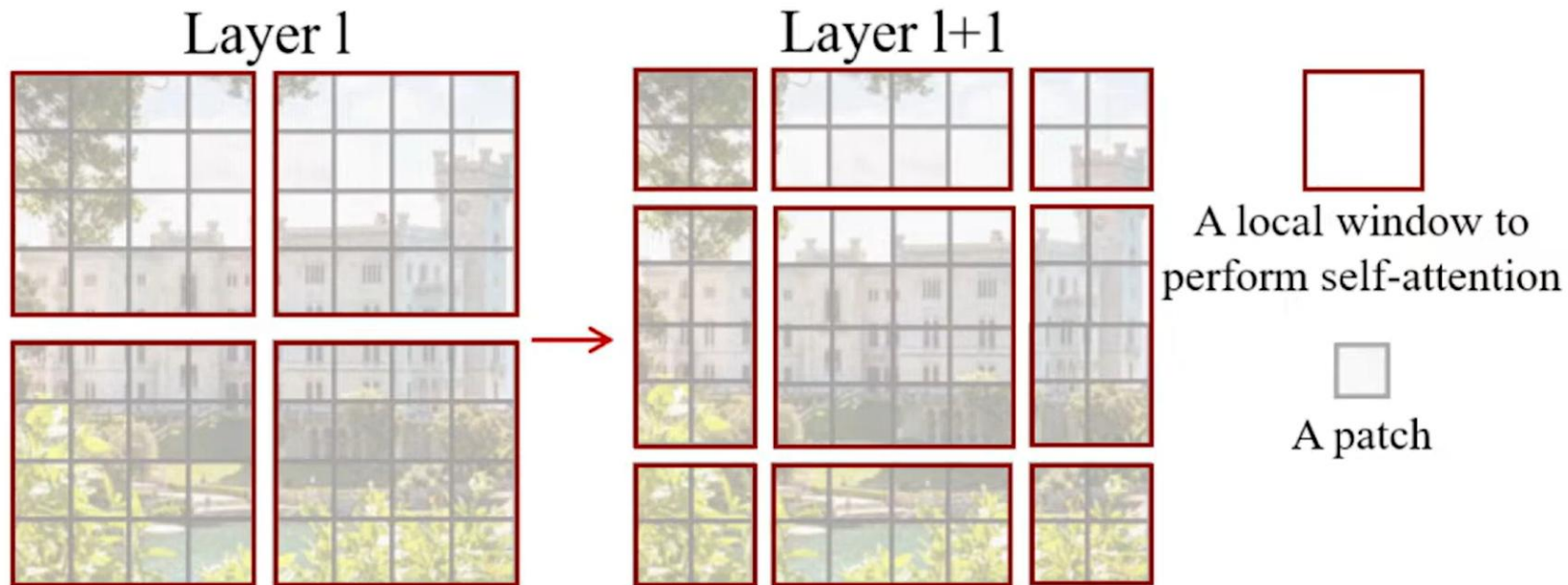
Swin Transformer Block : Relative Position Bias



6.3 Method

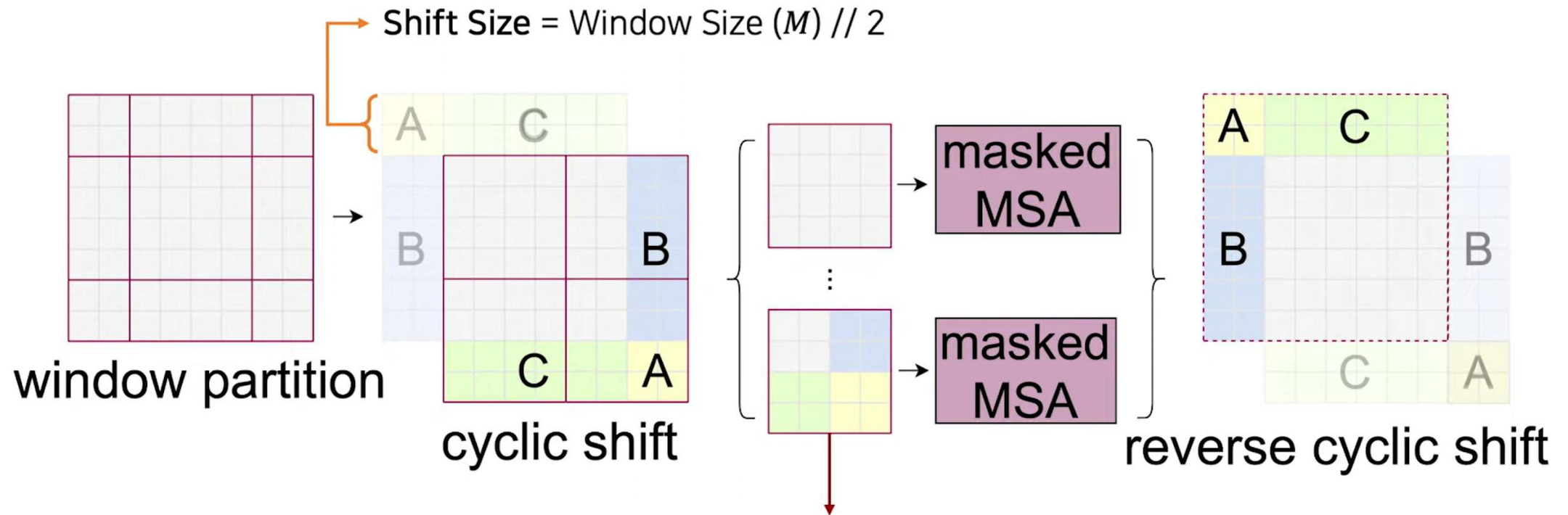
Swin Transformer Block : SW-MSA

- SW-MSA 수행 시 window 개수가 $\left(\frac{N_h}{M} + 1\right) \times \left(\frac{N_w}{M} + 1\right)$
- 기존보다 더 많은 윈도우가 생겨서 더 비효율적인 연산 과정이 될 수 있음
- Cyclic Shift & Attention Mask를 통해 W-MSA와 동일한 window 개수 사용



6.3 Method

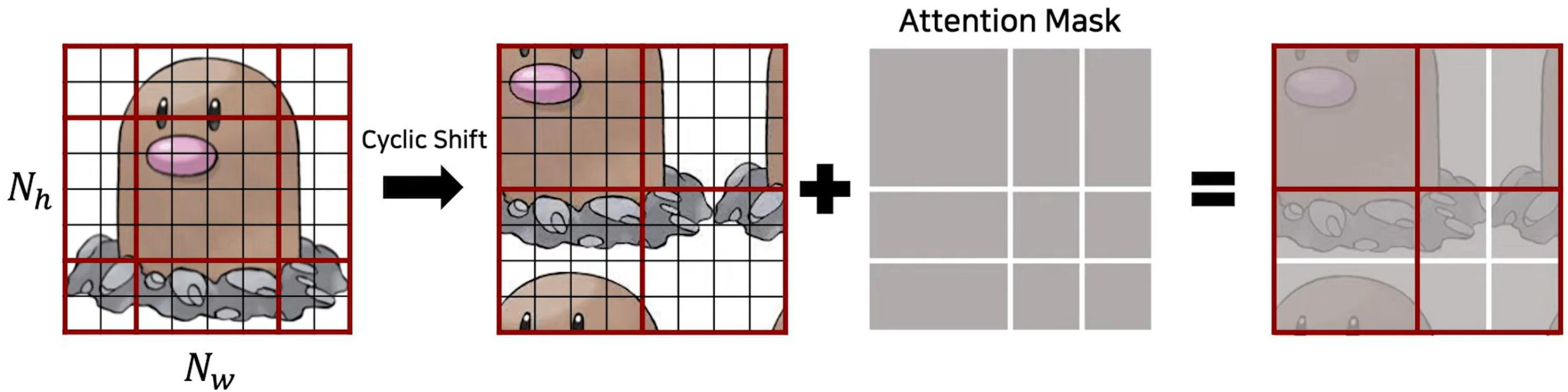
Swin Transformer Block : SW-MSA



서로 인접한 위치가 아니었기 때문에 Mask를 적용하여 mask 부분에서만 self attention을 적용

6.3 Method

Swin Transformer Block : SW-MSA



Chapter VII

SwinT 실험 결과 및 한계

6.1 SwinT 실험 결과

모델 버전

- Image Resolution (Default): 224 x 224
- 384 x 384 일 때는 window size 12 사용

	downsp. rate (output size)	Swin-T	Swin-S	Swin-B	Swin-L
stage 1	4× (56×56)	concat 4×4, 96-d, LN	concat 4×4, 96-d, LN	concat 4×4, 128-d, LN	concat 4×4, 192-d, LN
		$\begin{bmatrix} \text{win. sz. } 7 \times 7, \\ \text{dim 96, head 3} \end{bmatrix} \times 2$	$\begin{bmatrix} \text{win. sz. } 7 \times 7, \\ \text{dim 96, head 3} \end{bmatrix} \times 2$	$\begin{bmatrix} \text{win. sz. } 7 \times 7, \\ \text{dim 128, head 4} \end{bmatrix} \times 2$	$\begin{bmatrix} \text{win. sz. } 7 \times 7, \\ \text{dim 192, head 6} \end{bmatrix} \times 2$
stage 2	8× (28×28)	concat 2×2, 192-d, LN	concat 2×2, 192-d, LN	concat 2×2, 256-d, LN	concat 2×2, 384-d, LN
		$\begin{bmatrix} \text{win. sz. } 7 \times 7, \\ \text{dim 192, head 6} \end{bmatrix} \times 2$	$\begin{bmatrix} \text{win. sz. } 7 \times 7, \\ \text{dim 192, head 6} \end{bmatrix} \times 2$	$\begin{bmatrix} \text{win. sz. } 7 \times 7, \\ \text{dim 256, head 8} \end{bmatrix} \times 2$	$\begin{bmatrix} \text{win. sz. } 7 \times 7, \\ \text{dim 384, head 12} \end{bmatrix} \times 2$
stage 3	16× (14×14)	concat 2×2, 384-d, LN	concat 2×2, 384-d, LN	concat 2×2, 512-d, LN	concat 2×2, 768-d, LN
		$\begin{bmatrix} \text{win. sz. } 7 \times 7, \\ \text{dim 384, head 12} \end{bmatrix} \times 6$	$\begin{bmatrix} \text{win. sz. } 7 \times 7, \\ \text{dim 384, head 12} \end{bmatrix} \times 18$	$\begin{bmatrix} \text{win. sz. } 7 \times 7, \\ \text{dim 512, head 16} \end{bmatrix} \times 18$	$\begin{bmatrix} \text{win. sz. } 7 \times 7, \\ \text{dim 768, head 24} \end{bmatrix} \times 18$
stage 4	32× (7×7)	concat 2×2, 768-d, LN	concat 2×2, 768-d, LN	concat 2×2, 1024-d, LN	concat 2×2, 1536-d, LN
		$\begin{bmatrix} \text{win. sz. } 7 \times 7, \\ \text{dim 768, head 24} \end{bmatrix} \times 2$	$\begin{bmatrix} \text{win. sz. } 7 \times 7, \\ \text{dim 768, head 24} \end{bmatrix} \times 2$	$\begin{bmatrix} \text{win. sz. } 7 \times 7, \\ \text{dim 1024, head 32} \end{bmatrix} \times 2$	$\begin{bmatrix} \text{win. sz. } 7 \times 7, \\ \text{dim 1536, head 48} \end{bmatrix} \times 2$

Table 7. Detailed architecture specifications.

6.1 SwinT 실험 결과

Settings

- Classification에 ImageNet 1K로 실험
 - ImageNet-22K을 사전 학습에 사용
 - Object Detection에 COCO 2017 사용
 - Semantic Segmentation에 ADE20K 사용
-
- ✓ 다양한 Augmentation과 Regularization 필요
 - ✓ ViT에서 중요했던 repeated augmentation과 EMA는 필요하지 않았음
 - ✓ AdamW optimizer 사용

6.1 SwinT 실험 결과

Classification

ImageNet 1K

- DeiT vs Swin
- CNN based Models vs Swin
 - 성능과 학습 속도의 trade off가 더 작음

ImageNet 22K

- ViT vs Swin
 - 적은 파라미터로 더 높은 성능

method	image size	#param.	FLOPs	throughput (image / s)	ImageNet top-1 acc.
RegNetY-4G [47]	224 ²	21M	4.0G	1156.7	80.0
RegNetY-8G [47]	224 ²	39M	8.0G	591.6	81.7
RegNetY-16G [47]	224 ²	84M	16.0G	334.7	82.9
EffNet-B3 [57]	300 ²	12M	1.8G	732.1	81.6
EffNet-B4 [57]	380 ²	19M	4.2G	349.4	82.9
EffNet-B5 [57]	456 ²	30M	9.9G	169.1	83.6
EffNet-B6 [57]	528 ²	43M	19.0G	96.9	84.0
EffNet-B7 [57]	600 ²	66M	37.0G	55.1	84.3
ViT-B/16 [19]	384 ²	86M	55.4G	85.9	77.9
ViT-L/16 [19]	384 ²	307M	190.7G	27.3	76.5
DeiT-S [60]	224 ²	22M	4.6G	940.4	79.8
DeiT-B [60]	224 ²	86M	17.5G	292.3	81.8
DeiT-B [60]	384 ²	86M	55.4G	85.9	83.1
Swin-T	224 ²	29M	4.5G	755.2	81.3
Swin-S	224 ²	50M	8.7G	436.9	83.0
Swin-B	224 ²	88M	15.4G	278.1	83.3
Swin-B	384 ²	88M	47.0G	84.7	84.2

method	image size	#param.	FLOPs	throughput (image / s)	ImageNet top-1 acc.
R-101x3 [37]	384 ²	388M	204.6G	-	84.4
R-152x4 [37]	480 ²	937M	840.5G	-	85.4
ViT-B/16 [19]	384 ²	86M	55.4G	85.9	84.0
ViT-L/16 [19]	384 ²	307M	190.7G	27.3	85.2
Swin-B	224 ²	88M	15.4G	278.1	85.2
Swin-B	384 ²	88M	47.0G	84.7	86.0
Swin-L	384 ²	197M	103.9G	42.1	86.4

Table 1. Comparison of different backbones on ImageNet-1K classification. Throughput is measured using the GitHub repository of [65] and a V100 GPU, following [60].

6.1 SwinT 실험 결과

Classification

ImageNet 1K

- DeiT vs Swin
- CNN based Models vs Swin
 - 성능과 학습 속도의 trade off가 더 작음

ImageNet 22K

- ViT vs Swin
 - 적은 파라미터로 더 높은 성능

(a) Regular ImageNet-1K trained models

method	image size	#param.	FLOPs	throughput (image / s)	ImageNet top-1 acc.
RegNetY-4G [47]	224 ²	21M	4.0G	1156.7	80.0
RegNetY-8G [47]	224 ²	39M	8.0G	591.6	81.7
RegNetY-16G [47]	224 ²	84M	16.0G	334.7	82.9
EffNet-B3 [57]	300 ²	12M	1.8G	732.1	81.6
EffNet-B4 [57]	380 ²	19M	4.2G	349.4	82.9
EffNet-B5 [57]	456 ²	30M	9.9G	169.1	83.6
EffNet-B6 [57]	528 ²	43M	19.0G	96.9	84.0
EffNet-B7 [57]	600 ²	66M	37.0G	55.1	84.3
ViT-B/16 [19]	384 ²	86M	55.4G	85.9	77.9
ViT-L/16 [19]	384 ²	307M	190.7G	27.3	76.5
DeiT-S [60]	224 ²	22M	4.6G	940.4	79.8
DeiT-B [60]	224 ²	86M	17.5G	292.3	81.8
DeiT-B [60]	384 ²	86M	55.4G	85.9	83.1
Swin-T	224 ²	29M	4.5G	755.2	81.3
Swin-S	224 ²	50M	8.7G	436.9	83.0
Swin-B	224 ²	88M	15.4G	278.1	83.3
Swin-B	384 ²	88M	47.0G	84.7	84.2

(b) ImageNet-22K pre-trained models

method	image size	#param.	FLOPs	throughput (image / s)	ImageNet top-1 acc.
R-101x3 [37]	384 ²	388M	204.6G	-	84.4
R-152x4 [37]	480 ²	937M	840.5G	-	85.4
ViT-B/16 [19]	384 ²	86M	55.4G	85.9	84.0
ViT-L/16 [19]	384 ²	307M	190.7G	27.3	85.2
Swin-B	224 ²	88M	15.4G	278.1	85.2
Swin-B	384 ²	88M	47.0G	84.7	86.0
Swin-L	384 ²	197M	103.9G	42.1	86.4

Table 1. Comparison of different backbones on ImageNet-1K classification. Throughput is measured using the GitHub repository of [65] and a V100 GPU, following [60].

6.1 SwinT 실험 결과

Classification

ImageNet 1K

- DeiT vs Swin
- CNN based Models vs Swin
 - 성능과 학습 속도의 trade off가 더 작음

ImageNet 22K

- ViT vs Swin
 - 적은 파라미터로 더 높은 성능

(a) Regular ImageNet-1K trained models					
method	image size	#param.	FLOPs	throughput (image / s)	ImageNet top-1 acc.
RegNetY-4G [47]	224 ²	21M	4.0G	1156.7	80.0
RegNetY-8G [47]	224 ²	39M	8.0G	591.6	81.7
RegNetY-16G [47]	224 ²	84M	16.0G	334.7	82.9
EffNet-B3 [57]	300 ²	12M	1.8G	732.1	81.6
EffNet-B4 [57]	380 ²	19M	4.2G	349.4	82.9
EffNet-B5 [57]	456 ²	30M	9.9G	169.1	83.6
EffNet-B6 [57]	528 ²	43M	19.0G	96.9	84.0
EffNet-B7 [57]	600 ²	66M	37.0G	55.1	84.3
ViT-B/16 [19]	384 ²	86M	55.4G	85.9	77.9
ViT-L/16 [19]	384 ²	307M	190.7G	27.3	76.5
DeiT-S [60]	224 ²	22M	4.6G	940.4	79.8
DeiT-B [60]	224 ²	86M	17.5G	292.3	81.8
DeiT-B [60]	384 ²	86M	55.4G	85.9	83.1
Swin-T	224 ²	29M	4.5G	755.2	81.3
Swin-S	224 ²	50M	8.7G	436.9	83.0
Swin-B	224 ²	88M	15.4G	278.1	83.3
Swin-B	384 ²	88M	47.0G	84.7	84.2

(b) ImageNet-22K pre-trained models					
method	image size	#param.	FLOPs	throughput (image / s)	ImageNet top-1 acc.
R-101x3 [37]	384 ²	388M	204.6G	-	84.4
R-152x4 [37]	480 ²	937M	840.5G	-	85.4
ViT-B/16 [19]	384 ²	86M	55.4G	85.9	84.0
ViT-L/16 [19]	384 ²	307M	190.7G	27.3	85.2
Swin-B	224 ²	88M	15.4G	278.1	85.2
Swin-B	384 ²	88M	47.0G	84.7	86.0
Swin-L	384 ²	197M	103.9G	42.1	86.4

Table 1. Comparison of different backbones on ImageNet-1K classification. Throughput is measured using the GitHub repository of [65] and a V100 GPU, following [60].

6.1 SwinT 실험 결과

- ViT는 local window 개념이 없어서 inductive bias가 더 적음
- 사전학습 데이터가 많아질수록 성능 향상의 폭이 훨씬 큼
- 데이터가 많으면 많을수록 inductive bias는 적은 게 더 유리?
- 반대로 데이터가 적으면 어느 정도 inductive bias를 부여하는 걸 고려하는 게 중요할 것 같음

(a) Regular ImageNet-1K trained models					
method	image size	#param.	FLOPs	throughput (image / s)	ImageNet top-1 acc.
RegNetY-4G [47]	224 ²	21M	4.0G	1156.7	80.0
RegNetY-8G [47]	224 ²	39M	8.0G	591.6	81.7
RegNetY-16G [47]	224 ²	84M	16.0G	334.7	82.9
EffNet-B3 [57]	300 ²	12M	1.8G	732.1	81.6
EffNet-B4 [57]	380 ²	19M	4.2G	349.4	82.9
EffNet-B5 [57]	456 ²	30M	9.9G	169.1	83.6
EffNet-B6 [57]	528 ²	43M	19.0G	96.9	84.0
EffNet-B7 [57]	600 ²	66M	37.0G	55.1	84.3
ViT-B/16 [19]	384 ²	86M	55.4G	85.9	77.9
ViT-L/16 [19]	384 ²	307M	190.7G	27.3	76.5
DeiT-S [60]	224 ²	22M	4.6G	940.4	79.8
DeiT-B [60]	224 ²	86M	17.5G	292.3	81.8
DeiT-B [60]	384 ²	86M	55.4G	85.9	83.1
Swin-T	224 ²	29M	4.5G	755.2	81.3
Swin-S	224 ²	50M	8.7G	436.9	83.0
Swin-B	224 ²	88M	15.4G	278.1	83.3
Swin-B	384 ²	88M	47.0G	84.7	84.2
(b) ImageNet-22K pre-trained models					
method	image size	#param.	FLOPs	throughput (image / s)	ImageNet top-1 acc.
R-101x3 [37]	384 ²	388M	204.6G	-	84.4
R-152x4 [37]	480 ²	937M	840.5G	-	85.4
ViT-B/16 [19]	384 ²	86M	55.4G	85.9	84.0
ViT-L/16 [19]	384 ²	307M	190.7G	27.3	85.2
Swin-B	224 ²	88M	15.4G	278.1	85.2
Swin-B	384 ²	88M	47.0G	84.7	86.0
Swin-L	384 ²	197M	103.9G	42.1	86.4

Table 1. Comparison of different backbones on ImageNet-1K classification. Throughput is measured using the GitHub repository of [65] and a V100 GPU, following [60].

6.1 SwinT 실험 결과

Object Detection

Frameworks 기준

- ResNet-50 vs Swin
 - + 3.4 ~ 4.2 box AP

(a) Various frameworks							
Method	Backbone	AP ^{box}	AP ₅₀ ^{box}	AP ₇₅ ^{box}	#param.	FLOPs	FPS
Cascade	R-50	46.3	64.3	50.5	82M	739G	18.0
Mask R-CNN	Swin-T	50.5	69.3	54.9	86M	745G	15.3
ATSS	R-50	43.5	61.9	47.0	32M	205G	28.3
	Swin-T	47.2	66.5	51.3	36M	215G	22.3
RepPointsV2	R-50	46.5	64.6	50.3	42M	274G	13.6
	Swin-T	50.0	68.5	54.2	45M	283G	12.0
Sparse R-CNN	R-50	44.5	63.4	48.2	106M	166G	21.0
	Swin-T	47.9	67.3	52.3	110M	172G	18.4

6.1 SwinT 실험 결과

Backbone 기준

- CNN based Model vs Swin
- 현재 기준 6위 (2021.07.27)
 - ❖ 현재 SOTA 60.1 box AP
- DeiT는 Vit와 거의 동일한 구조
 - ✓ 모든 layer가 동일한 해상도
 - ✓ Deconvolution layer를 사용하여 다른 해상도 추출

(a) Various frameworks							
Method	Backbone	AP ^{box}	AP ^{box} ₅₀	AP ^{box} ₇₅	#param.	FLOPs	FPS
Cascade	R-50	46.3	64.3	50.5	82M	739G	18.0
Mask R-CNN	Swin-T	50.5	69.3	54.9	86M	745G	15.3
ATSS	R-50	43.5	61.9	47.0	32M	205G	28.3
	Swin-T	47.2	66.5	51.3	36M	215G	22.3
RepPointsV2	R-50	46.5	64.6	50.3	42M	274G	13.6
	Swin-T	50.0	68.5	54.2	45M	283G	12.0
Sparse R-CNN	R-50	44.5	63.4	48.2	106M	166G	21.0
	Swin-T	47.9	67.3	52.3	110M	172G	18.4

(b) Various backbones w. Cascade Mask R-CNN							
	AP ^{box}	AP ^{box} ₅₀	AP ^{box} ₇₅	AP ^{mask}	AP ^{mask} ₅₀	AP ^{mask} ₇₅	paramFLOPsFPS
DeiT-S [†]	48.0	67.2	51.7	41.4	64.2	44.3	80M 889G 10.4
R50	46.3	64.3	50.5	40.1	61.7	43.4	82M 739G 18.0
Swin-T	50.5	69.3	54.9	43.7	66.6	47.1	86M 745G 15.3
X101-32	48.1	66.5	52.4	41.6	63.9	45.2	101M 819G 12.8
Swin-S	51.8	70.4	56.3	44.7	67.9	48.5	107M 838G 12.0
X101-64	48.3	66.4	52.3	41.7	64.0	45.1	140M 972G 10.4
Swin-B	51.9	70.9	56.5	45.0	68.4	48.7	145M 982G 11.6

(c) System-level Comparison						
Method	mini-val		test-dev		#param. FLOPs	
	AP ^{box}	AP ^{mask}	AP ^{box}	AP ^{mask}		
RepPointsV2* [11]	-	-	52.1	-	-	-
GCNet* [6]	51.8	44.7	52.3	45.4	-	1041G
RelationNet++* [12]	-	-	52.7	-	-	-
SpineNet-190 [20]	52.6	-	52.8	-	164M	1885G
ResNeSt-200* [75]	52.5	-	53.3	47.1	-	-
EfficientDet-D7 [58]	54.4	-	55.1	-	77M	410G
DetectoRS* [45]	-	-	55.7	48.5	-	-
YOLOv4 P7* [3]	-	-	55.8	-	-	-
Copy-paste [25]	55.9	47.2	56.0	47.4	185M	1440G
X101-64 (HTC++)	52.3	46.0	-	-	155M	1033G
Swin-B (HTC++)	56.4	49.1	-	-	160M	1043G
Swin-L (HTC++)	57.1	49.5	57.7	50.2	284M	1470G
Swin-L (HTC++)*	58.0	50.4	58.7	51.1	284M	-

Table 2. Results on COCO object detection and instance segmentation. [†]denotes that additional deconvolution layers are used to produce hierarchical feature maps. * indicates multi-scale testing.

6.1 SwinT 실험 결과

Semantic Segmentation

- SETR + T-Large vs UperNet + Swin-L
 - 적은 파라미터로 더 높은 성능 향상
 - 현재 순위 7위 (2021.07.27)
 - ❖ SOTA 57.0 mIoU

ADE20K		val	test	#param.	FLOPs	FPS
Method	Backbone	mIoU	score			
DANet [22]	ResNet-101	45.2	-	69M	1119G	15.2
DLab.v3+ [10]	ResNet-101	44.1	-	63M	1021G	16.0
ACNet [23]	ResNet-101	45.9	38.5	-		
DNL [68]	ResNet-101	46.0	56.2	69M	1249G	14.8
OCRNet [70]	ResNet-101	45.3	56.0	56M	923G	19.3
UperNet [66]	ResNet-101	44.9	-	86M	1029G	20.1
OCRNet [70]	HRNet-w48	45.7	-	71M	664G	12.5
DLab.v3+ [10]	ResNeSt-101	46.9	55.1	66M	1051G	11.9
DLab.v3+ [10]	ResNeSt-200	48.4	-	88M	1381G	8.1
SETR [78]	T-Large [†]	50.3	61.7	308M	-	-
UperNet	DeiT-S [†]	44.0	-	52M	1099G	16.2
UperNet	Swin-T	46.1	-	60M	945G	18.5
UperNet	Swin-S	49.3	-	81M	1038G	15.2
UperNet	Swin-B [‡]	51.6	-	121M	1841G	8.7
UperNet	Swin-L [‡]	53.5	62.8	234M	3230G	6.2

Table 3. Results of semantic segmentation on the ADE20K val and test set. [†] indicates additional deconvolution layers are used to produce hierarchical feature maps. [‡] indicates that the model is pre-trained on ImageNet-22K.

6.1 SwinT 실험 결과

Relative Positional Bias 여부 & Shifted Window 여부

- Window 간 관계를 학습하는 것이 Detection과 Segmentation 성능 향상에 도움

	ImageNet		COCO		ADE20k
	top-1	top-5	AP ^{box}	AP ^{mask}	mIoU
w/o shifting	80.2	95.1	47.7	41.5	43.3
shifted windows	81.3	95.6	50.5	43.7	46.1
no pos.	80.1	94.9	49.2	42.6	43.8
abs. pos.	80.5	95.2	49.0	42.4	43.2
abs.+rel. pos.	81.3	95.6	50.2	43.4	44.0
rel. pos. w/o app.	79.3	94.7	48.2	41.9	44.1
rel. pos.	81.3	95.6	50.5	43.7	46.1

Table 4. Ablation study on the *shifted windows* approach and different position embedding methods on three benchmarks, using the Swin-T architecture. w/o shifting: all self-attention modules adopt regular window partitioning, without *shifting*; abs. pos.: absolute position embedding term of ViT; rel. pos.: the default settings with an additional relative position bias term (see Eq. (4)); app.: the first scaled dot-product term in Eq. (4).

6.1 SwinT 실험 결과

Relative Positional Bias 여부 & Shifted Window 여부

- Dense Prediction의 경우 성능이 크게 향상

	ImageNet		COCO		ADE20k
	top-1	top-5	AP ^{box}	AP ^{mask}	mIoU
w/o shifting	80.2	95.1	47.7	41.5	43.3
shifted windows	81.3	95.6	50.5	43.7	46.1
no pos.	80.1	94.9	49.2	42.6	43.8
abs. pos.	80.5	95.2	49.0	42.4	43.2
abs.+rel. pos.	81.3	95.6	50.2	43.4	44.0
rel. pos. w/o app.	79.3	94.7	48.2	41.9	44.1
rel. pos.	81.3	95.6	50.5	43.7	46.1

Table 4. Ablation study on the *shifted windows* approach and different position embedding methods on three benchmarks, using the Swin-T architecture. w/o shifting: all self-attention modules adopt regular window partitioning, without *shifting*; abs. pos.: absolute position embedding term of ViT; rel. pos.: the default settings with an additional relative position bias term (see Eq. (4)); app.: the first scaled dot-product term in Eq. (4).

6.1 SwinT 실험 결과

Window 적용 방법에 따른 속도 비교

method	MSA in a stage (ms)				Arch. (FPS)		
	S1	S2	S3	S4	T	S	B
sliding window (naive)	122.5	38.3	12.1	7.6	183	109	77
sliding window (kernel)	7.6	4.7	2.7	1.8	488	283	187
Performer [13]	4.8	2.8	1.8	1.5	638	370	241
window (w/o shifting)	2.8	1.7	1.2	0.9	770	444	280
shifted window (padding)	3.3	2.3	1.9	2.2	670	371	236
shifted window (cyclic)	3.0	1.9	1.3	1.0	755	437	278

Table 5. Real speed of different self-attention computation methods and implementations on a V100 GPU.

6.1 SwinT 실험 결과

Sliding Window와 Shifted Window 비교

	Backbone	ImageNet		COCO		ADE20k
		top-1	top-5	AP ^{box}	AP ^{mask}	mIoU
sliding window	Swin-T	81.4	95.6	50.2	43.5	45.8
Performer [13]	Swin-T	79.0	94.2	-	-	-
shifted window	Swin-T	81.3	95.6	50.5	43.7	46.1

Table 6. Accuracy of Swin Transformer using different methods for self-attention computation on three benchmarks.

Summary

- *It is our belief that a **unified architecture across computer vision and natural language processing** could benefit both fields, since it would **facilitate joint modeling of visual and textual signals** and **the modeling knowledge from both domains can be more deeply shared**. [1] p.2*
- NLP와 컴퓨터 비전에 유사한 아키텍처를 사용하는 것은 연구 속도를 크게 가속화할 수 있음. 물론 트랜스포머는 여전히 많은 데이터에 의존하는 구조임.
- 그럼에도 불구하고, 이것은 양 분야 모두에 있어 분명한 발전의 한 걸음.
- Local window를 적용해서 inductive bias 를 개입
- Patch merging 을 통해 hierarchical 구조 형성
- 이미지 특성을 고려한 모델 구조
- 다양한 vision task에 적합한 구조
- ViT보다 적은 Computation Complexity

Summary

- 로컬 윈도우 내에서만 Self-Attention을 수행 (지역 정보 위주의 표현)
 - Swin T는 전체 이미지가 아니라 고정된 윈도우(슬라이딩 윈도우) 내에서만 self-attention을 계산함. 이로 인해 윈도우 간의 연결성이 부족해져, 서로 다른 윈도우에 위치한 정보가 직접적으로 교류되지 못하는 한계가 있음. 윈도우의 경계에 위치한 패치는 실제로 인접해 있음에도 불구하고 동일한 윈도우에 속하지 않으면 self-attention에서 분리되어 계산된다는 점도 단점.
- 글로벌 컨텍스트 통합의 제한
 - 일반적인 ViT는 글로벌 self-attention으로 이미지 전체의 정보를 한 번에 통합할 수 있지만, SwinT는 주로 로컬 정보를 효과적으로 처리하도록 설계(local window self-attention)되어 글로벌 관계를 포착하는 능력이 상대적으로 약할 수 있습니다. 윈도우를 여러 단계에 걸쳐 이동(shifted window)하여 어느 정도 정보를 섞지만, 근본적으로 global attention의 직접성과는 거리감이 있음. 따라서 멀리 떨어진 객체 간의 상호작용을 모델링하기엔 부족할 수 있음.
- 윈도우 크기, 구조에 따른 하이퍼파라미터 의존성
 - 윈도우의 크기, 병합 단계(patch merging) 등 구조적 하이퍼파라미터에 성능이 좌우되며, 다양한 해상도나 비정형 이미지 입력에 대해 완전히 유연하게 적용되기는 어려움. CNN이 다양한 해상도 변화를 자연스럽게 소화하는 것과 달리, Swin 구조는 입력 스케일과 구조 디자인의 영향을 많이 받음.
- 경계 부근 아티팩트 및 정보 손실 위험
 - 윈도우 기반 연산에서 윈도우 경계에 있는 패치들은 인접 윈도우 패치와 직접적으로 상호작용하지 못하기 때문에, 경계에서 정보 손실이나 아티팩트가 발생할 수 있음. 이를 완전히 해소하려면 오버랩이나 별도의 윈도우 설계가 추가되어야 하며, 복잡도가 늘 수 있음.
- 복잡한 구조적 변화 또는 확장에는 한계
 - Transformer 계열이 가진 구조적 확장성(CNN의 다채로운 커널, 합성곱 연산 등)과 비교해, SwinT는 윈도우 기반 트랜스포머라는 특성상 구조적 다양화나 커스텀화가 다소 제한적일 수 있음.
 - 기본적으로 Swin은 윈도우 기반 attention이기 때문에, ViT처럼 진정한 전역 attention을 학습하기 어려움. 특히 transformer가 가지는 global dependency modeling이라는 본래 장점을 Swin은 완전히 살리지 못함. → **ViT-L이나 DeiT 같은 구조보다 더 "local inductive bias"를 가짐.(by local window)**
- 따라서 Swin Transformer는 계산 효율성과 성능 면에서는 상당한 강점을 지니지만, 윈도우 단위로 attention을 제한하는 디자인상의 구조로 인해 글로벌 관계 포착의 한계, 경계에서의 정보 손실, 구조적 유연성 부족 등의 단점을 갖고 있습니다.

Chapter VIII

Questions

Reference

Reference

- ViT : <https://arxiv.org/pdf/2010.11929>
- SwinT : <https://arxiv.org/pdf/2103.14030>
- Vision Transformer(ViT)에 대한 시각적 설명 : <https://discuss.pytorch.kr/t/vision-transformer-a-visual-guide-to-vision-transformers/4158>
- Georgia Tech's Transformers continued & Vision Transformers : https://faculty.cc.gatech.edu/~zk15/teaching/AY2025_cs7643_spring/assets/L14_Transformers_VisionTransformers.pdf
- Blogs : <https://velog.io/@imfromk/CV-An-Image-Is-Worth-16x16-Words-Transformers-for-Image-Recognition-At-ScaleViT-review>, <https://velog.io/@hsbc/Vision-Transformer>, <https://ssongs.tistory.com/57>, <https://daeba27.tistory.com/108>, <https://m.blog.naver.com/nueyet/222984347342>, <https://csm-kr.tistory.com/54>, <https://kimjy99.github.io/%EB%85%BC%EB%AC%B8%EB%A6%AC%EB%B7%B0/swin-transformer/>, <https://velog.io/@hsbc/Vision-Transformer>,
- 서울대학교 산업공학과 DSBA 연구실 : https://www.youtube.com/watch?v=2lZvuU_IIMA&t=1873s
- ViT & SwinT 설명 : <https://www.louisbouchard.ai/will-transformers-replace-cnns-for-vision/>